

T.R.
GEBZE TECHNICAL UNIVERSITY
FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING

**LOGIC-HALT: A HYBRID METAMORPHIC
TESTING AND INFORMATION THEORY
ALGORITHM FOR HALLUCINATION
DETECTION IN LARGE LANGUAGE MODELS**

ÖMER FARUK ŞENER

**SUPERVISOR
DR. ÖĞR. ÜYESİ TÜLAY AYYILDIZ**

**GEBZE
2026**

T.R.
GEBZE TECHNICAL UNIVERSITY
FACULTY OF ENGINEERING
COMPUTER ENGINEERING DEPARTMENT

**LOGIC-HALT: A HYBRID
METAMORPHIC TESTING AND
INFORMATION THEORY ALGORITHM
FOR HALLUCINATION DETECTION IN
LARGE LANGUAGE MODELS**

ÖMER FARUK ŞENER

SUPERVISOR
DR. ÖĞR. ÜYESİ TÜLAY AYYILDIZ

2026
GEBZE

	GRADUATION PROJECT JURY APPROVAL FORM
---	--

This study has been accepted as an Undergraduate Graduation Project in the Department of Computer Engineering on January 2026 by the following jury.

JURY

Member

(Supervisor) : Dr. Öğr. Üyesi Tülay AYYILDIZ

Member : Doç. Dr. Habil KALKAN

ABSTRACT

Large Language Models (LLMs) have demonstrated remarkable capabilities in natural language processing tasks. However, they are prone to generating fluent but factually incorrect or logically inconsistent responses, a phenomenon known as “hallucination.” Detecting hallucinations without access to ground-truth answers remains a fundamental challenge, as the most informative detection signals—such as comparing outputs against known correct answers—are inherently unavailable during real-world inference.

This thesis presents LOGIC-HALT (LOGical Inconsistency and Compression-based HALlucination Testing), a multi-signal hallucination detection framework that operates entirely without ground-truth answers at inference time. The system combines five complementary signals through a Bayesian-optimized fusion layer: (1) self-verification, where the model is asked to fact-check its own answer via Natural Language Inference (NLI), (2) cross-response inconsistency measured through pairwise NLI contradiction analysis using DeBERTa-v3-large (435M parameters), (3) token entropy computed from log-probabilities, (4) pairwise Normalized Compression Distance (NCD) between response variants, and (5) answer-level minority penalty from majority voting across extracted answers. These signals are generated by querying the target LLM with semantically equivalent question variants produced through metamorphic transformations—including paraphrasing, logical negation, variable substitution, premise reordering, and redundant context injection—each validated for semantic preservation via Sentence-BERT cosine similarity.

A critical contribution of this work is the identification and correction of a ground truth leakage problem in the original evaluation framework. The initial system incorporated a Ground Truth Contradiction signal that compared model outputs against known correct answers, accounting for 63.8% of the fusion weight and producing artificially inflated metrics (Precision = 0.977). Since this signal is unavailable at inference time, we replaced it with inference-time proxy signals and re-optimized all hyperparameters using Bayesian optimization (Optuna TPE sampler, 500 trials, 13 hyperparameters).

Evaluation on the TruthfulQA benchmark (817 questions) yields an F1 score of 0.727 (Precision = 0.602, Recall = 0.917) under precision-balanced optimization in fully reference-free conditions. When optimized purely for F1 without precision constraints, the system achieves F1 = 0.735 with perfect recall but reduced precision (0.581). A systematic ablation study across 8 signal combinations reveals that answer-level majority voting is the strongest individual signal (39.19% optimized weight), and

that Negation Contradiction is the only additional signal that improves precision (0.578 $\rightarrow$ 0.606). The system is implemented with GPU optimization for NVIDIA RTX 5070 (12 GB VRAM, CUDA 12.8) with FP16 inference and batch NLI processing, and includes a Flask-based web interface for real-time analysis.

The findings demonstrate that while multi-signal consistency analysis achieves high recall for hallucination screening, the fundamental limitation of consistency-based approaches—that consistency does not imply correctness—caps precision at approximately 60% without external knowledge sources. This establishes a clear direction for future work integrating retrieval-augmented verification and cross-model consistency signals.

Keywords: Large Language Models, Hallucination Detection, Metamorphic Testing, Natural Language Inference, Information Theory, Bayesian Optimization, TruthfulQA, Normalized Compression Distance.

ÖZET

Büyük Dil Modelleri (BDM'ler), doğal dil işleme görevlerinde olağanüstü yetenekler sergilemektedir. Ancak bu modeller, akıcı fakat olgusal olarak yanlış veya mantıksal olarak tutarsız yanıtlar üretmeye eğilimlidir; bu durum “halüsinasyon” olarak adlandırılmaktadır. Dış bilgi tabanlarına karşı doğrulanabilen olgusal halüsinasyonların aksine, mantıksal görevlerdeki muhakeme halüsinasyonları geleneksel doğrulama yöntemleriyle kolayca tespit edilememektedir.

Bu tez, mantıksal muhakeme problemlerine verilen BDM yanıtlarındaki halüsinasyonları tespit etmek için referanssız (ground-truth bağımsız) yeni bir çerçeve olan LOGIC-HALT'ı (LOGical Inconsistency and Compression-based HALlucination Testing) sunmaktadır. Önerilen yaklaşım üç tamamlayıcı tekniği birleştirmektedir: (1) anlamsal olarak eşdeğer soru varyantları üretmek için metamorfik test, (2) graf tabanlı çelişki tespiti yoluyla yanıt tutarlılığını analiz etmek için Doğal Dil Çıkarımı (NLI) modelleri ve (3) yanıt belirsizliğini ve karmaşıklığını ölçmek için token entropisi ve Normalize Edilmiş Sıkıştırma Mesafesi (NCD) dahil bilgi teorisi metrikleri.

LOGIC-HALT sistemi iki aşamalı bir tespit hattı uygulamaktadır. İlk aşamada, birden fazla yanıt arasında nihai cevaplar çıkarılarak ve karşılaştırılarak cevap düzeyinde tutarlılık değerlendirilmektedir. İkinci aşamada, NLI tabanlı tutarlılık puanlaması, token log-olasılıklarından entropi ölçümü ve sıkıştırma tabanlı karmaşıklık analizi kullanılarak detaylı analiz gerçekleştirilmektedir. Bu metrikler, nihai bir halüsinasyon risk skoru üretmek için ağırlıklı bir füzyon katmanı aracılığıyla birleştirilmektedir.

Olgusal, matematiksel, bilimsel ve mantıksal kategorileri kapsayan 20 çeşitli sorudan oluşan özel bir veri seti üzerinde yapılan deneysel değerlendirme, sistemin güvenilir ve halüsinasyonlu yanıtları ayırt etme kapasitesini göstermektedir. Sistem, açıklama stili farklılıklarının neden olduğu yanlış pozitifleri azaltırken gerçek halüsinasyonlara duyarlılığı koruyarak, iki aşamalı yaklaşım sayesinde geliştirilmiş kesinlik elde etmektedir.

Bu çalışmanın katkıları şunlardır: (1) açık uçlu muhakeme görevleri için uygun, ground-truth bağımsız tespit metodolojisi, (2) BDM değerlendirmesi için metamorfik test ilkelerinin bilgi teorisi ile entegrasyonu ve (3) gerçek zamanlı halüsinasyon tespiti için web tabanlı arayüze sahip pratik bir uygulama. Bulgular, tutarlılık analizinin bilgi teorisi ölçümleriyle birleştirilmesinin, harici bilgi kaynakları gerektirmeden BDM güvenilirliğini değerlendirmek için sağlam bir yaklaşım sağladığını göstermektedir.

Anahtar Kelimeler: Büyük Dil Modelleri, Halüsinasyon Tespiti, Metamorfik Test, Doğal Dil Çıkarımı, Bilgi Teorisi, Normalize Edilmiş Sıkıştırma Mesafesi.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my advisor, Dr. Öğr. Üyesi Tülay AYYILDIZ, for her invaluable guidance, continuous support, and encouragement throughout this graduation project. Her expertise and insights have been instrumental in shaping this research.

I am also grateful to Doç. Dr. Habil KALKAN for serving on my thesis committee and providing constructive feedback that helped improve this work.

I would like to thank the faculty members of the Computer Engineering Department at Gebze Technical University for providing a stimulating academic environment and the necessary resources for conducting this research.

Special thanks to my family and friends for their unwavering support and understanding during the completion of this project.

Finally, I acknowledge the open-source community and the developers of the tools and libraries used in this project, including Hugging Face Transformers, PyTorch, and Flask, which made this implementation possible.

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
AUC-ROC	Area Under the Receiver Operating Characteristic Curve
FN	False Negative
FP	False Positive
GPT	Generative Pre-trained Transformer
LLM	Large Language Model
LOGIC-HALT	LOGical Inconsistency and Compression-based HALlucination Testing
MR	Metamorphic Relation
MT	Metamorphic Testing
NCD	Normalized Compression Distance
NLI	Natural Language Inference
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
TN	True Negative
TP	True Positive
UI	User Interface

CONTENTS

Abstract	iv
Özet	vi
Acknowledgements	vii
List of Abbreviations	viii
Contents	xii
List of Figures	xiii
List of Tables	xiv
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Proposed Solution: LOGIC-HALT	3
1.4 Contributions	5
1.5 Thesis Organization	6
2 Related Work	8
2.1 Hallucination in Large Language Models	8
2.1.1 Definition and Types of Hallucination	8
2.1.2 Causes of Hallucination	8
2.2 Existing Hallucination Detection Methods	9
2.2.1 Retrieval-Based Methods	9
2.2.2 Self-Consistency Methods	9
2.2.3 Uncertainty-Based Methods	10
2.3 Metamorphic Testing	10
2.3.1 Principles of Metamorphic Testing	10
2.3.2 Application to Machine Learning	10
2.4 Information Theory in Text Analysis	11
2.4.1 Kolmogorov Complexity	11
2.4.2 Normalized Compression Distance	11
2.4.3 Entropy in Language Models	11

2.5	Natural Language Inference	12
2.6	Bayesian Hyperparameter Optimization	12
2.7	Hallucination Benchmarks	13
2.8	Summary and Research Gap	13
3	Methodology	14
3.1	System Overview	14
3.1.1	Design Evolution: From Ground-Truth Dependence to Inference-Time Detection	15
3.2	Module A: The Morpher (Variant Generation)	16
3.2.1	Transformation Operators	16
3.2.2	Teacher LLM for Variant Generation	17
3.2.3	Semantic Filtering via Sentence-BERT	17
3.3	Module B: The Interrogator (Response Collection)	18
3.3.1	Target LLM and Parameters	18
3.3.2	Hybrid Response Format: ANSWER + REASONING	19
3.3.3	Response Data Structure	19
3.3.4	Self-Verification	19
3.4	Module C: The Consistency Engine (NLI-Based Analysis)	20
3.4.1	NLI Model	20
3.4.2	Pairwise Contradiction Matrix Construction	20
3.4.3	Graph-Based Consistency Metrics	21
3.4.4	Consistency Score	22
3.4.5	Self-Verification Score	22
3.5	Module D: The Complexity Engine (Information-Theoretic Analysis)	23
3.5.1	Token Entropy	23
3.5.2	Entropy Normalization	23
3.5.3	Pairwise Normalized Compression Distance (NCD)	23
3.5.4	NCD Normalization	24
3.6	Module F: The Answer Validator (Answer-Level Consistency)	24
3.6.1	Answer Extraction	24
3.6.2	Majority Voting	25
3.6.3	Minority Penalty	25
3.7	Module E: The Fusion Layer (Five-Signal Decision)	26
3.7.1	Five-Signal Fusion Formula	26
3.7.2	Bayesian Weight Optimization	26
3.7.3	Optimized Parameters	27
3.7.4	Classification Decision	28
3.7.5	Confidence Estimation	28

3.8	Complete Algorithm	28
3.9	Computational Complexity Analysis	29
3.10	Summary	29
4	Experimental Setup	32
4.1	Implementation	32
4.1.1	Technology Stack	32
4.1.2	System Architecture	32
4.1.3	NLI Model Selection	34
4.1.4	Labeling Model	34
4.1.5	Target LLM	35
4.2	Datasets	36
4.2.1	Primary Dataset: TruthfulQA	36
4.2.2	Pilot Dataset	37
4.2.3	Data Collection via Batch API	37
4.3	Evaluation Metrics	38
4.3.1	Classification Metrics	38
4.3.2	Confusion Matrix	38
4.3.3	Optimization Objective	39
4.4	Experimental Configuration	39
4.4.1	Five-Signal Fusion Parameters	39
4.4.2	NLI Consistency Parameters	40
4.4.3	Complexity Engine Parameters	40
4.4.4	Answer Validator Parameters	41
4.4.5	Morpher Transformation Parameters	41
4.5	Bayesian Optimization	42
4.5.1	Optimization Framework	42
4.5.2	Search Space	42
4.5.3	Pre-Computation Strategy	43
4.6	Hardware Environment	44
4.7	Ablation Study Design	44
4.8	Web Interface	45
4.9	Experimental Procedure	46
4.10	Reproducibility	47
4.11	Summary	47
5	Results and Discussion	49
5.1	Ground Truth Contradiction: Discovery and Correction	49
5.1.1	Impact of the Correction	49
5.2	Five-Signal System Results	50

5.2.1	Standard F1 Optimization	50
5.2.2	Precision-Balanced Optimization	50
5.3	Bayesian Optimization Results	51
5.3.1	Optimized Signal Weights	51
5.3.2	Optimization Convergence	52
5.4	Ablation Study	52
5.4.1	Ablation Analysis	52
5.5	Threshold Analysis	53
5.6	Signal Importance Analysis	54
5.6.1	Why Minority Penalty Dominates	54
5.6.2	The Consistency-Correctness Gap	54
5.7	Discussion	55
5.7.1	LOGIC-HALT as a Screening Tool	55
5.7.2	Comparison with Related Methods	55
5.8	Limitations	56
5.9	Summary	56
6	Conclusion	58
6.1	Summary of Contributions	58
6.2	Key Findings	59
6.3	Limitations	59
6.4	Future Work	60
6.4.1	Short-Term Improvements	60
6.4.2	Medium-Term Extensions	60
6.4.3	Long-Term Research Directions	61
6.5	Concluding Remarks	61
	Bibliography	63
	Curriculum Vitae	64
	Appendices	66

LIST OF FIGURES

3.1	High-level architecture of the LOGIC-HALT system. The pipeline flows from variant generation through response collection, multi-signal analysis, and fusion to a final hallucination decision.	15
3.2	The LOGIC-HALT hallucination detection algorithm. The pipeline proceeds through seven phases: variant generation, response collection, self-verification, consistency analysis, complexity analysis, answer validation, and five-signal fusion.	30

LIST OF TABLES

3.1	Bayesian-optimized fusion weights and threshold (500 TPE trials, 817 TruthfulQA questions). All weights are normalized to sum to 1.0. . . .	27
4.1	Software dependencies and minimum versions.	33
4.2	Source modules and their architectural roles.	33
4.3	NLI model configuration.	34
4.4	LLM interrogation parameters.	36
4.5	TruthfulQA dataset overview.	36
4.6	Pilot dataset composition by category.	37
4.7	Confusion matrix structure for hallucination detection.	39
4.8	Optimized five-signal fusion weights (500 trials, 817 questions). . . .	40
4.9	NLI consistency engine parameters (Bayesian-optimized).	40
4.10	Complexity engine parameters.	41
4.11	Answer validator parameters.	41
4.12	Morpher (Module A) configuration.	42
4.13	Bayesian optimization configuration.	42
4.14	Hyperparameter search space.	43
4.15	Hardware configuration.	44
4.16	Ablation study signal combinations.	45
5.1	Impact of removing ground truth dependency.	49
5.2	Five-signal system performance (F1-optimized, 500 trials).	50
5.3	Precision-balanced optimization results (1000 trials).	51
5.4	Optimized 5-signal fusion weights (500 trials, TruthfulQA).	51
5.5	Optimization convergence.	52
5.6	Ablation study results (sorted by F1, 500 trials each).	53
5.7	Performance at different thresholds (optimized weights).	54
6.1	Sample questions from the test dataset.	66
6.2	API endpoints.	67

1. INTRODUCTION

1.1. Background and Motivation

Large Language Models (LLMs) such as GPT-4 [openai2023gpt4](#), Claude, and Llama have fundamentally transformed the landscape of natural language processing and artificial intelligence. Built upon the Transformer architecture [vaswani2017attention](#) and trained on vast corpora of text data, these models demonstrate remarkable capabilities across a wide range of tasks, including question answering, logical reasoning, summarization, and code generation [1]. Their outputs are often fluent, coherent, and stylistically indistinguishable from human-written text, which has led to their rapid adoption in both academic and industrial applications.

However, alongside these impressive capabilities lies a fundamental and well-documented limitation: LLMs *hallucinate*. Hallucination refers to the tendency of language models to generate text that is fluent and confident yet factually incorrect, logically inconsistent, or entirely fabricated [2]. Unlike human experts who typically express uncertainty or qualify their statements when unsure, LLMs present hallucinated content with the same surface-level confidence as veridical information, making it exceedingly difficult for end users to distinguish reliable outputs from fabricated ones [3]. This problem has been identified as one of the most critical barriers to the trustworthy deployment of LLMs in high-stakes domains such as healthcare, legal reasoning, financial analysis, and autonomous decision-making [kaur2022trustworthy](#).

The severity of the hallucination problem is amplified by its heterogeneous nature. Factual hallucinations—where the model produces incorrect dates, names, or statistics—can potentially be verified against external knowledge bases such as Wikipedia. Reasoning hallucinations, however, are fundamentally more challenging to detect. In such cases, the model may produce a perfectly grammatical and seemingly logical argument that nevertheless contains a subtle but critical logical flaw. For instance, an LLM might correctly state the premises of a syllogism but arrive at an invalid conclusion, or it might introduce an unstated assumption that invalidates the entire chain of reasoning [liu2023evaluating](#). Because the “truth” in reasoning tasks depends on logical inference rather than factual lookup, external knowledge bases offer no recourse, and the correctness of such outputs can only be assessed through internal analysis of the reasoning structure itself.

Furthermore, hallucination rates remain disturbingly high even in state-of-the-art models. Studies have shown that modern LLMs can hallucinate on a significant fraction of queries, particularly on questions designed to elicit common misconceptions,

false beliefs, or logically tricky formulations [4]. The TruthfulQA benchmark, which comprises 817 questions specifically designed to probe such failure modes, has demonstrated that even the most capable models frequently produce confident but incorrect responses across categories including health, law, finance, and common misconceptions. This underscores the urgent need for automated, scalable, and reliable methods to detect hallucinations without requiring human verification for every output.

Existing approaches to hallucination detection broadly fall into two paradigms. Retrieval-based methods, such as FActScore [5], decompose generated text into atomic claims and verify each claim against external knowledge sources. While effective for factual verification, these methods inherently depend on the availability of comprehensive and up-to-date knowledge bases, and they are inapplicable to reasoning tasks where correctness cannot be looked up. Self-consistency methods, such as SelfCheckGPT [3], sample multiple responses to the same prompt and measure agreement across samples under the intuition that factual knowledge is reproducibly recalled. However, these methods can be defeated by *consistent hallucinations*—situations where the model repeatedly produces the same incorrect answer with high confidence across all samples, yielding high apparent consistency despite factual error.

These limitations motivate the development of a more robust, multi-signal approach that combines complementary detection strategies. The ideal system would operate without requiring ground-truth answers at inference time, would be resilient to consistent hallucinations, and would provide interpretable risk scores rather than binary decisions. This thesis addresses these requirements through the design, implementation, and evaluation of LOGIC-HALT.

1.2. Problem Statement

The core problem addressed in this thesis is the **reference-free detection of hallucinations** in LLM responses without relying on ground-truth answers, external knowledge bases, or human annotations at inference time. Specifically, we aim to answer the following research question:

Can we reliably detect hallucinations in LLM responses by fusing multiple complementary signals—including self-verification, cross-response consistency, information-theoretic complexity, and answer-level agreement—through an automatically optimized decision framework?

This problem poses several interconnected challenges:

1. **The Oracle Problem:** In logical reasoning and open-ended question answering, the correct answer may not be readily available for comparison. Any detection system must therefore operate without ground-truth references during inference,

relying solely on properties of the model’s own outputs.

2. **Consistent Hallucinations:** A model that consistently produces the same incorrect answer across multiple queries cannot be detected by consistency analysis alone. Detection must incorporate additional signals that capture uncertainty or structural anomalies even when surface-level responses appear uniform.
3. **Style-Induced False Positives:** When comparing multiple LLM responses, natural variation in explanation style, word choice, and reasoning presentation can be mistaken for semantic inconsistency by Natural Language Inference (NLI) models. A detection system must distinguish genuine contradictions from superficial stylistic differences.
4. **Metric Insufficiency:** No single metric—whether consistency, entropy, compression distance, or answer agreement—is individually sufficient for reliable hallucination detection. Each captures different aspects of model behavior, and an effective system must combine them in a principled manner.
5. **Hyperparameter Sensitivity:** Multi-signal fusion systems are sensitive to the relative weighting of their component signals and to decision thresholds. Manual tuning is unlikely to find optimal configurations, especially across diverse question types and difficulty levels.
6. **Ground Truth Leakage:** A critical but subtle pitfall in hallucination detection research is the inadvertent use of ground-truth information during evaluation in ways that would not be available during real-world inference. If a signal relies on comparing model outputs to known correct answers, the reported performance may be artificially inflated and unattainable in practice.

Our approach addresses each of these challenges through a combination of metamorphic testing principles, multi-signal analysis, two-phase detection, and Bayesian hyperparameter optimization, as detailed in the following section.

1.3. Proposed Solution: LOGIC-HALT

This thesis presents LOGIC-HALT (**LOG**ical **I**nconsistency and **C**ompression-based **H**ALLucination **T**esting), a multi-signal hallucination detection framework that combines five complementary signals through a Bayesian-optimized fusion layer. The system is designed as a modular, six-stage pipeline consisting of the following components:

1. **Module A – The Morpher (Variant Generation):** Generates semantically equivalent variants of the input question using LLM-guided metamorphic transformations—including syntactic paraphrasing, logical equivalence (contrapositive), variable substitution, premise reordering, and redundant context injection. Each variant is validated through Sentence-BERT **reimers2019sentence** cosine similarity to ensure semantic preservation above a configurable threshold.
2. **Module B – The Interrogator (Response Collection):** Queries the target LLM with both the original question and all generated variants, collecting structured responses in a HYBRID format that explicitly separates the final answer from the supporting reasoning. Token-level log-probabilities are captured alongside each response for subsequent entropy analysis.
3. **Module C – The Consistency Engine (NLI Analysis):** Evaluates semantic consistency across responses using a pre-trained Natural Language Inference model (DeBERTa-v3 [11]). Pairwise NLI classifications are used to construct a contradiction matrix, from which graph-based consistency metrics—including contradiction density, clustering coefficient, and average contradiction strength—are derived. Crucially, NLI comparison is performed on the extracted *answer portions* of responses rather than full texts, mitigating false positives caused by explanation style variation. A self-verification signal is also computed by asking the model to verify its own initial answer through an additional NLI-based consistency check.
4. **Module D – The Complexity Engine (Information Theory):** Computes two information-theoretic metrics that capture orthogonal aspects of response quality. Token entropy, derived from log-probabilities, measures the model’s internal uncertainty during generation [4]. Normalized Compression Distance (NCD), computed as pairwise compression-based similarity across responses [9], captures structural redundancy and coherence patterns. Both metrics are normalized to $[0, 1]$ through calibrated transformations.
5. **Module E – The Fusion Layer (Bayesian-Optimized Decision):** Combines the five signals—Self-Verification, Inconsistency, Entropy, NCD, and Minority Penalty—through a weighted linear combination to produce a continuous hallucination risk score. The weights and decision threshold are optimized using Bayesian optimization via the Tree-structured Parzen Estimator (TPE) sampler, implemented through the Optuna framework with 500 optimization trials.
6. **Module F – The Answer Validator (Majority Voting):** Extracts final answers from all collected responses using pattern matching, performs majority voting

to identify the consensus answer, and applies a configurable minority penalty to responses that disagree with the majority. This module provides both a fast-track classification path (when answer agreement exceeds 80%, the question is classified as safe without full analysis) and an additional signal for the fusion layer.

The complete detection pipeline implements a two-phase strategy. In the first phase, the Answer Validator performs a rapid answer-level consistency check. If sufficient agreement is found among extracted answers, the system returns a low-risk classification immediately, avoiding the computational overhead of full NLI and complexity analysis. In the second phase, triggered when answer-level consistency is below the fast-track threshold, the system performs detailed multi-signal analysis through the Consistency Engine, Complexity Engine, and self-verification, combining all signals in the Fusion Layer for a final decision.

A critical design decision in LOGIC-HALT concerns the elimination of ground-truth dependency. During the development of this system, we discovered that the original evaluation framework incorporated a Ground Truth (GT) Contradiction signal that compared model outputs against known correct answers using NLI. While this signal was highly informative during evaluation (accounting for over 60% of the fusion weight in the original 4-signal configuration), it is fundamentally unavailable during real-world inference, where the correct answer is not known. Retaining this signal would yield performance metrics that are misleadingly optimistic and unattainable in practice. We therefore replaced the GT Contradiction signal with two inference-time-available proxy signals—Self-Verification (where the model verifies its own answer) and Minority Penalty (from answer-level majority voting)—and re-optimized all hyperparameters from scratch. The resulting 5-signal system reports honest metrics that reflect true deployment performance.

1.4. Contributions

The main contributions of this thesis are as follows:

1. **A Reference-Free, Multi-Signal Detection Framework:** We propose LOGIC-HALT, a hallucination detection framework that operates entirely without ground-truth answers at inference time. The system combines five complementary signals—Self-Verification, Inconsistency, Entropy, NCD, and Minority Penalty—through a weighted fusion layer, providing more robust detection than any single-signal approach. To our knowledge, this is the first system to combine metamorphic testing, NLI-based consistency analysis, information-theoretic complexity

measures, and answer-level majority voting in a unified pipeline for hallucination detection.

2. **Identification and Correction of Ground Truth Leakage:** We identify a critical methodological issue in multi-signal hallucination detection: the inadvertent use of ground-truth information as a detection signal during evaluation, which inflates reported metrics to levels unattainable during real-world inference. We correct this by replacing the ground-truth-dependent signal with inference-time proxy signals and re-optimizing the entire system, yielding honest and reproducible performance metrics.
3. **Bayesian Hyperparameter Optimization:** We demonstrate the effectiveness of Bayesian optimization using the Tree-structured Parzen Estimator (TPE) for jointly optimizing 13 hyperparameters across all system components. Over 500 optimization trials on the TruthfulQA benchmark (817 questions), we achieve an F1 score of 0.727 with Precision = 0.602 and Recall = 0.917 under a precision-balanced objective, representing a practical operating point for reference-free deployment.
4. **Two-Phase Detection Pipeline:** We introduce an efficient two-phase detection strategy that first performs a fast answer-level consistency check using majority voting before proceeding to computationally expensive NLI and information-theoretic analysis. This design reduces both computational cost and false positives caused by explanation style variation, while maintaining high recall for genuine hallucinations.
5. **Comprehensive Ablation Study:** We conduct a systematic ablation study across 8 signal combinations to quantify the contribution of each signal to overall detection performance. This analysis reveals the relative importance of each component and validates the necessity of multi-signal fusion.
6. **Practical GPU-Optimized Implementation:** We provide a complete, modular implementation evaluated on consumer-grade hardware (NVIDIA RTX 5070 with 12 GB VRAM, CUDA 12.8), demonstrating that effective hallucination detection is achievable without enterprise-grade computational resources. The system includes a web-based interface for real-time analysis and is designed for extensibility to additional LLM providers and signal types.

1.5. Thesis Organization

The remainder of this thesis is organized as follows:

- **Chapter 2: Related Work** reviews existing approaches to hallucination detection in LLMs, including retrieval-based methods, self-consistency methods, and uncertainty-based methods. It also covers metamorphic testing principles, Natural Language Inference, and information-theoretic approaches to text analysis, establishing the theoretical foundations upon which LOGIC-HALT is built.
- **Chapter 3: Methodology** presents the detailed architecture of the LOGIC-HALT system, describing the six modules (Morpher, Interrogator, Consistency Engine, Complexity Engine, Fusion Layer, and Answer Validator), their mathematical formulations, the five-signal fusion mechanism, and the Bayesian optimization procedure.
- **Chapter 4: Experimental Setup** describes the implementation details, including the technology stack, NLI model selection, and target LLM configuration. It details the evaluation on the TruthfulQA benchmark (817 questions), defines the evaluation metrics, and specifies the hardware environment (RTX 5070, CUDA 12.8) and experimental parameters.
- **Chapter 5: Results and Discussion** presents the experimental results, including overall performance metrics ($F1 = 0.727$, $Precision = 0.602$, $Recall = 0.917$), the ablation study across 8 signal combinations, threshold analysis, and a detailed discussion of the ground truth leakage issue and its resolution.
- **Chapter 6: Conclusion** summarizes the findings and contributions, discusses limitations of the current approach, and outlines directions for future research including cross-model generalization, domain-specific fine-tuning, and integration with external knowledge sources.

2. RELATED WORK

This chapter reviews the relevant literature in three main areas: hallucination detection methods for Large Language Models, metamorphic testing in software engineering, and information-theoretic approaches to text analysis.

2.1. Hallucination in Large Language Models

2.1.1. Definition and Types of Hallucination

Hallucination in LLMs refers to the generation of content that is fluent and seemingly plausible but factually incorrect, logically inconsistent, or unsupported by the input context [2]. The phenomenon has been categorized into several types:

- **Factual Hallucination:** Generation of incorrect facts, such as wrong dates, names, or statistics. For example, stating that “Albert Einstein won the Nobel Prize in Physics in 1905” when it was actually awarded in 1921.
- **Reasoning Hallucination:** Logical errors in multi-step reasoning tasks where the model produces an incorrect conclusion despite appearing to follow logical steps. This is particularly problematic because the output may “look” correct while containing fundamental logical flaws.
- **Intrinsic Hallucination:** Content that contradicts the source material or input context.
- **Extrinsic Hallucination:** Content that cannot be verified from the source material, neither supported nor contradicted.

Understanding these distinctions is crucial because different types of hallucination may require different detection strategies. Our work primarily focuses on reasoning hallucinations in logical tasks, which cannot be easily verified against external knowledge bases.

2.1.2. Causes of Hallucination

Several factors contribute to hallucination in LLMs [4]:

1. **Training Data Issues:** Models may learn incorrect information present in training data, or memorize patterns that do not generalize correctly.
2. **Probabilistic Nature:** LLMs generate text by predicting the most likely next token, which optimizes for fluency rather than factual accuracy.
3. **Knowledge Cutoff:** Models have a training cutoff date and cannot access information beyond that point.
4. **Reasoning Limitations:** Despite impressive performance, LLMs struggle with complex logical reasoning, often failing on problems requiring multiple inference steps.

2.2. Existing Hallucination Detection Methods

2.2.1. Retrieval-Based Methods

Retrieval-based approaches verify LLM outputs against external knowledge sources. FactScore [5] decomposes generated text into atomic facts and verifies each against Wikipedia. While effective for factual claims, this approach has limitations:

- Requires comprehensive, up-to-date knowledge bases
- Cannot handle reasoning tasks where truth depends on logical inference
- May miss hallucinations about obscure topics not covered in knowledge bases

2.2.2. Self-Consistency Methods

SelfCheckGPT [3] proposes sampling multiple responses to the same question and measuring consistency. The intuition is that factual information should be consistently reproduced, while hallucinated content varies across samples. The method computes several consistency metrics:

$$\text{BERTScore}(r_1, r_2) = \frac{\sum_i \max_j \cos(h_{r_1}^i, h_{r_2}^j)}{|r_1|} \quad (2.1)$$

where h_r^i represents BERT embeddings of tokens in response r .

However, self-consistency methods can fail when models produce “consistent hallucinations” – repeatedly generating the same incorrect answer with high confidence. This is particularly common for questions where the model has learned incorrect patterns during training.

2.2.3. Uncertainty-Based Methods

Some approaches leverage model uncertainty signals for hallucination detection. Token-level entropy, computed from log-probabilities, can indicate when a model is uncertain about its predictions:

$$H(r) = -\frac{1}{T} \sum_{t=1}^T \log p(w_t | w_{<t}) \quad (2.2)$$

High entropy suggests the model is uncertain, which may correlate with hallucination. However, models can also confidently produce incorrect outputs, limiting the effectiveness of uncertainty-based methods alone.

2.3. Metamorphic Testing

2.3.1. Principles of Metamorphic Testing

Metamorphic Testing (MT) is a software testing technique that addresses the “oracle problem” – the challenge of determining expected outputs for complex systems [6]. Instead of comparing outputs to expected values, MT verifies that outputs satisfy certain relationships (Metamorphic Relations) across transformed inputs.

Definition 2.1 (Metamorphic Relation). *Given a function f and input transformation T , a metamorphic relation MR specifies the expected relationship between $f(x)$ and $f(T(x))$.*

For example, if f computes the sine function, and $T(x) = \pi - x$, then the metamorphic relation states that $f(x) = f(T(x))$ (since $\sin(x) = \sin(\pi - x)$).

2.3.2. Application to Machine Learning

MT has been successfully applied to test machine learning systems [7], including:

- Image classifiers: Testing that classification is invariant to certain transformations (rotation, scaling)
- Search engines: Verifying that adding irrelevant terms does not change result rankings
- Machine translation: Checking consistency across paraphrased inputs

Our work extends MT principles to LLM evaluation by generating semantically equivalent question variants and checking response consistency. The key insight is that a reliable model should produce consistent answers across different phrasings of the same logical problem.

2.4. Information Theory in Text Analysis

2.4.1. Kolmogorov Complexity

Kolmogorov complexity $K(x)$ measures the length of the shortest program that produces string x [8]. While theoretically powerful, Kolmogorov complexity is uncomputable. In practice, compression algorithms provide approximations.

2.4.2. Normalized Compression Distance

The Normalized Compression Distance (NCD) uses compression as a proxy for Kolmogorov complexity [9]:

$$\text{NCD}(x, y) = \frac{C(xy) - \min(C(x), C(y))}{\max(C(x), C(y))} \quad (2.3)$$

where $C(x)$ denotes the compressed size of x using algorithms like gzip or zlib. NCD has been applied to:

- Plagiarism detection
- Author identification
- Malware classification
- Clustering of biological sequences

In our work, we hypothesize that hallucinated responses may exhibit abnormal compression characteristics – either unusually repetitive (low NCD) or lacking coherent structure (high NCD compared to correct responses).

2.4.3. Entropy in Language Models

Information entropy quantifies uncertainty in a probability distribution. For language models, token-level entropy measures how uncertain the model is about each prediction:

$$H_t = - \sum_{v \in V} p(v|w_{<t}) \log p(v|w_{<t}) \quad (2.4)$$

where V is the vocabulary and $w_{<t}$ are previous tokens. Average entropy across a response provides a global uncertainty measure:

$$\bar{H} = \frac{1}{T} \sum_{t=1}^T H_t \quad (2.5)$$

Research suggests that hallucinated content often exhibits different entropy patterns compared to factual content, though the relationship is complex and context-dependent.

2.5. Natural Language Inference

Natural Language Inference (NLI) models classify the logical relationship between a premise and hypothesis into three categories: *Entailment*, *Contradiction*, or *Neutral* [10].

State-of-the-art NLI models like DeBERTa [11] achieve high accuracy on benchmark datasets and can be applied to detect semantic contradictions between LLM responses. Given two responses R_i and R_j to semantically equivalent questions, a contradiction label suggests inconsistency that may indicate hallucination.

2.6. Bayesian Hyperparameter Optimization

Multi-signal detection systems require careful calibration of component weights and decision thresholds. Manual tuning is impractical when the search space spans more than a handful of parameters. Bayesian optimization addresses this by building a probabilistic model of the objective function and using it to select promising configurations.

The Tree-structured Parzen Estimator (TPE), as implemented in the Optuna framework, has proven effective for hyperparameter optimization in machine learning systems. Unlike grid search or random search, TPE models the distribution of good and bad parameter configurations separately, enabling efficient exploration of high-dimensional search spaces with relatively few evaluations. This approach is particularly well-suited to our problem, where 13 hyperparameters (signal weights, decision thresholds, NLI parameters, normalization bounds) must be jointly optimized to maximize detection performance.

2.7. Hallucination Benchmarks

The TruthfulQA benchmark comprises 817 questions specifically designed to elicit false beliefs and common misconceptions from language models. Questions span categories including health, law, finance, politics, and conspiracies, making it a comprehensive testbed for hallucination detection systems. Each question includes a ground-truth answer and a set of common incorrect answers, enabling systematic evaluation of both LLM truthfulness and detection system performance.

Other relevant benchmarks include HaluEval, which provides fine-grained hallucination annotations, and FELM, which focuses on faithfulness evaluation. In this thesis, we adopt TruthfulQA as the primary evaluation benchmark due to its size (817 questions), diversity, and widespread adoption in the hallucination detection literature.

2.8. Summary and Research Gap

Existing hallucination detection methods have several limitations:

1. Retrieval-based methods cannot handle reasoning tasks without external knowledge bases
2. Self-consistency methods fail on “consistent hallucinations” where the model repeatedly produces the same incorrect answer
3. Uncertainty methods alone are insufficient as models can confidently hallucinate
4. Most methods rely on a single detection signal, limiting robustness
5. Few methods address the ground truth leakage problem—where evaluation metrics are inflated by signals that require known correct answers unavailable at inference time

LOGIC-HALT addresses these gaps by combining five complementary signals: metamorphic testing (to expose inconsistencies through semantic variants), NLI-based consistency analysis (to detect contradictions), information-theoretic metrics (to capture uncertainty and complexity), answer-level majority voting (to detect answer disagreement), and self-verification (to probe logical coherence). These signals are combined through a Bayesian-optimized fusion layer, and the system is explicitly designed to operate without ground-truth answers at inference time. This multi-signal, reference-free approach provides a more robust framework for hallucination detection than any single-method solution.

3. METHODOLOGY

This chapter presents the complete architecture of the LOGIC-HALT framework, describing each module’s functionality, the mathematical formulations governing its operation, and the overall detection pipeline. LOGIC-HALT is a modular, reference-free hallucination detection system that operates entirely at inference time, requiring no access to ground-truth answers or external knowledge bases. The system integrates metamorphic testing, natural language inference, information-theoretic analysis, and answer-level validation into a unified five-signal fusion framework whose weights are optimized via Bayesian hyperparameter search.

3.1. System Overview

LOGIC-HALT is structured as a six-module pipeline, illustrated in Figure 3.1. The core insight underlying the framework is that a reliable LLM, when asked semantically equivalent variants of the same question, should produce consistent answers; hallucinated responses, by contrast, tend to exhibit instability under semantic perturbation. The system exploits this principle through five complementary detection signals that capture different facets of hallucination.

1. **Module A – The Morpher:** Generates semantically equivalent question variants using five transformation operators and a teacher LLM.
2. **Module B – The Interrogator:** Queries the target LLM with the original question and all variants, collecting structured responses with log-probabilities, and performs self-verification.
3. **Module C – The Consistency Engine:** Builds a pairwise contradiction matrix using bidirectional NLI and derives graph-based consistency metrics.
4. **Module D – The Complexity Engine:** Computes token entropy from log-probabilities and pairwise Normalized Compression Distance (NCD) across responses.
5. **Module E – The Fusion Layer:** Combines five signals via a weighted linear model with Bayesian-optimized weights.
6. **Module F – The Answer Validator:** Extracts final answers via regex, performs majority voting, and applies a minority penalty to outlier responses.

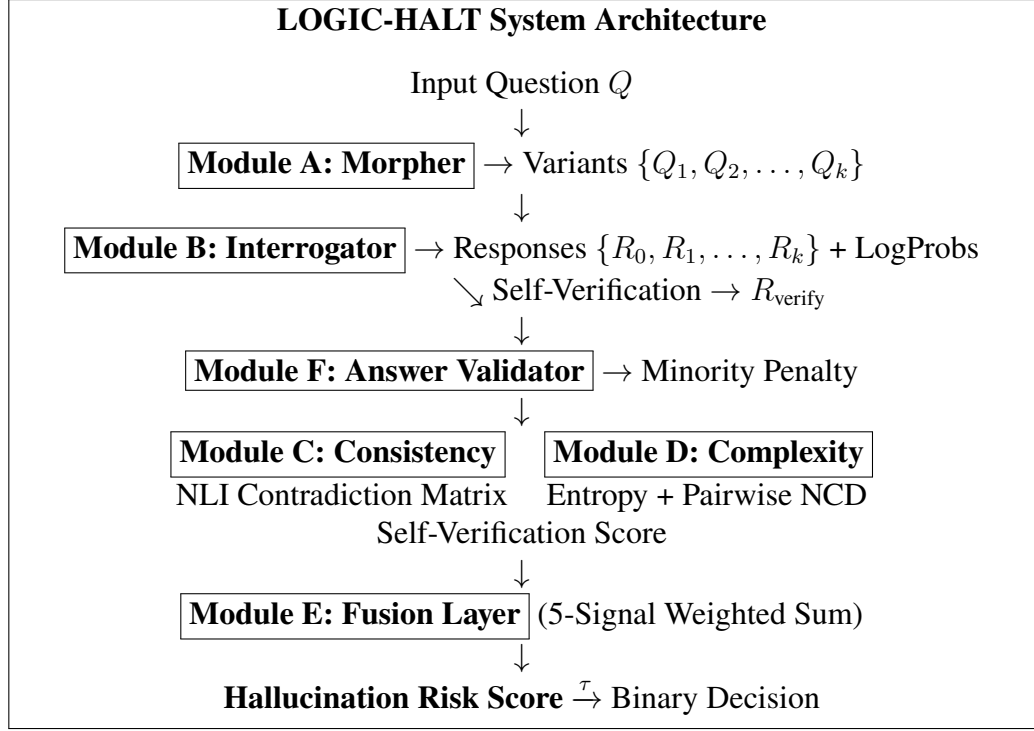


Figure 3.1: High-level architecture of the LOGIC-HALT system. The pipeline flows from variant generation through response collection, multi-signal analysis, and fusion to a final hallucination decision.

3.1.1. Design Evolution: From Ground-Truth Dependence to Inference-Time Detection

An important aspect of the LOGIC-HALT design process warrants explicit discussion. The system was originally conceived with six detection signals, including a *ground-truth contradiction* signal that compared each LLM response against a known correct answer using NLI. During initial development and optimization, this ground-truth (GT) signal was the strongest individual predictor of hallucination, and the system achieved an F1 score of 0.7730 on TruthfulQA when this signal was included.

However, this design contained a fundamental flaw: at inference time, the ground-truth answer is precisely what is unknown. A hallucination detector that requires the correct answer to function is, by definition, unusable in production. Upon recognizing this circularity, we undertook a complete redesign of the fusion layer, removing the GT contradiction signal and replacing it with five signals that are computable solely from the LLM’s own outputs. This redesign required re-optimization of all weights using Bayesian search (500 Optuna TPE trials over 817 TruthfulQA questions), yielding a revised F1 score of 0.7346. While modestly lower than the GT-inclusive configuration, this system is *honest* and *deployable*: every signal it uses is available at inference time without any oracle knowledge.

Throughout this chapter, all formulations describe the final, inference-time-only five-signal system unless explicitly stated otherwise.

3.2. Module A: The Morpher (Variant Generation)

The Morpher generates semantically equivalent variants of the input question using a teacher LLM guided by meta-prompts. The goal is to probe the target LLM from multiple angles: if the model truly “understands” the question, its answers should remain stable across logically equivalent reformulations. The Morpher is grounded in the principles of metamorphic testing **chen2018metamorphic**, where a system’s correctness is assessed not by checking individual outputs against expected values, but by verifying that relationships between outputs of related inputs are preserved.

3.2.1. Transformation Operators

We define five transformation operators $\{T_1, T_2, T_3, T_4, T_5\}$ that preserve semantic equivalence while altering surface-level characteristics of the question:

1. **Paraphrase** (T_1): Restructures the sentence using different words and syntactic constructions while maintaining the identical logical meaning.
 - Original: “All men are mortal.”
 - Transformed: “Every human being will eventually die.”
2. **Negation / Contrapositive** (T_2): Applies logically equivalent negation transformations such as double negation or the contrapositive.

$$P \rightarrow Q \equiv \neg Q \rightarrow \neg P \quad (3.1)$$

- Original: “If it rains, the ground gets wet.”
 - Transformed: “If the ground is not wet, it did not rain.”
3. **Variable Substitution** (T_3): Replaces entity names (people, objects, locations) with abstract variables or different names while preserving the logical structure.
 - Original: “Socrates is a man. All men are mortal.”
 - Transformed: “Entity X belongs to category Y. All members of Y are Z.”
 4. **Premise Reordering** (T_4): Changes the presentation order of assumptions and premises without altering the logical dependencies among them.

- Original: “All men are mortal. Socrates is a man. Is Socrates mortal?”
- Transformed: “Socrates is a man. All men are mortal. Is Socrates mortal?”

5. **Redundant Context (T_5):** Adds irrelevant background information that should not affect the answer if the model correctly identifies the relevant premises.

- Original: “What is $2 + 2$?”
- Transformed: “On a sunny Tuesday afternoon in Istanbul, what is $2 + 2$?”

Each transformation type probes a different aspect of the model’s robustness:

- T_1 (Paraphrase) tests whether the model is sensitive to surface-level word choice.
- T_2 (Negation) tests logical equivalence understanding.
- T_3 (Variable Substitution) tests whether the model’s reasoning is bound to specific entities or is genuinely structural.
- T_4 (Premise Reordering) tests sensitivity to presentation order.
- T_5 (Redundant Context) tests susceptibility to irrelevant distractors.

3.2.2. Teacher LLM for Variant Generation

Variant generation is performed by a *teacher LLM* (GPT-4o-mini) using structured meta-prompts. For each transformation type T_j , a dedicated prompt template instructs the teacher model to produce a variant of the input question while adhering to the rules of that transformation. The teacher LLM is called with temperature 0.7 and a maximum output length of 500 tokens. Its output is expected in JSON format:

```
{"variant": "<transformed question>", "type": "<transformation_type>"}
```

The system generates $k = 5$ variants (one per transformation type) by default. Combined with the original question, this yields $k + 1 = 6$ question instances to be queried.

3.2.3. Semantic Filtering via Sentence-BERT

Not all generated variants successfully preserve semantic equivalence; some may drift too far from the original meaning due to imperfect LLM generation. To filter out such cases, we employ Sentence-BERT **reimers2019sentence** using the **all-MiniLM-L6-v2** model, which produces 384-dimensional sentence embeddings.

For each candidate variant Q_i , we compute cosine similarity with the original question Q :

$$\text{sim}(Q, Q_i) = \frac{\mathbf{e}_Q \cdot \mathbf{e}_{Q_i}}{\|\mathbf{e}_Q\| \cdot \|\mathbf{e}_{Q_i}\|} \quad (3.2)$$

where $\mathbf{e}_Q, \mathbf{e}_{Q_i} \in \mathbb{R}^{384}$ are the respective sentence embeddings. Variants whose similarity falls below the threshold $\tau_{\text{sim}} = 0.75$ are discarded and excluded from subsequent pipeline stages:

$$Q_i \text{ is valid} \iff \text{sim}(Q, Q_i) \geq \tau_{\text{sim}} = 0.75 \quad (3.3)$$

This threshold was determined empirically to balance two competing objectives: retaining enough diversity to probe the model effectively while ensuring that the question semantics are genuinely preserved.

Definition 3.1 (Metamorphic Relation). *Given the original question Q and a transformation operator T_j , the generated variant $Q_i = T_j(Q)$ satisfies the metamorphic relation if and only if the correct answer to Q_i is identical to the correct answer to Q . Formally:*

$$\forall T_j \in \{T_1, \dots, T_5\}, \quad \text{Answer}(Q) = \text{Answer}(T_j(Q)) \quad (3.4)$$

A reliable LLM should therefore produce the same answer for both Q and $T_j(Q)$; deviations constitute evidence of potential hallucination.

3.3. Module B: The Interrogator (Response Collection)

The Interrogator queries the target LLM with the original question and all valid variants, collecting structured responses that separate the answer from its reasoning, along with token-level log-probabilities required by the Complexity Engine.

3.3.1. Target LLM and Parameters

The target LLM in our experimental setup is GPT-4o-mini, queried via the OpenAI API with the following fixed parameters:

- **Temperature:** 0.3 (low, to encourage more deterministic outputs while allowing some variation)
- **Maximum tokens:** 400
- **Log-probabilities:** Enabled, with top-5 logprobs returned per token
- **Samples per question:** 1 response per question instance

The choice of a low temperature (0.3) is deliberate. A high temperature would introduce artificial variation that obscures the signal we seek: variation arising from the model’s genuine uncertainty or logical inconsistency. By constraining the sampling, we ensure that observed inconsistencies across variants are more likely to reflect genuine comprehension failures rather than random sampling noise.

3.3.2. Hybrid Response Format: ANSWER + REASONING

A critical design decision in LOGIC-HALT is the adoption of a *hybrid response format* that explicitly separates the model’s answer from its reasoning. The prompt instructs the target LLM to respond in the following structured format:

ANSWER: [Direct answer in 1-2 sentences]

REASONING: [Brief explanation in 2-3 sentences]

This separation serves a specific purpose in the downstream consistency analysis. When comparing responses to semantically equivalent questions using NLI, we compare *only the ANSWER portions*, not the full responses. This prevents false positives caused by explanation style variance: two responses may express the same answer using entirely different explanatory approaches, which a full-text NLI comparison might incorrectly classify as contradictory. Entropy and NCD computation, however, use the full response text.

3.3.3. Response Data Structure

Each response R_i from the target LLM is stored with its associated metadata:

$$R_i = \{\text{text}_i, \text{logprobs}_i, \text{answer}_i, \text{reasoning}_i, \text{variant_type}_i\} \quad (3.5)$$

where text_i is the full response text, $\text{logprobs}_i = \{\log p(w_t \mid w_{<t})\}_{t=1}^{T_i}$ is the sequence of token log-probabilities, answer_i and reasoning_i are the parsed portions, and $\text{variant_type}_i \in \{\text{original}, T_1, T_2, T_3, T_4, T_5\}$ indicates which transformation produced the input question.

3.3.4. Self-Verification

In addition to collecting responses to all variants, the Interrogator performs a *self-verification* step. After obtaining the original response R_0 to the original question Q , the system asks the same model (or optionally a stronger model) to fact-check its own answer:

Question: {Q}

Someone gave this answer: {R_0.answer}

Carefully fact-check this answer. Is it factually correct or does it contain errors?

ANSWER: State CORRECT or INCORRECT

REASONING: Explain why in 1-2 sentences

The self-verification query is issued at temperature 0.0 (fully deterministic) to obtain the model’s most confident assessment. The resulting verification response R_{verify} is then compared against the original answer using NLI (described in Section 3.4.5) to produce a self-verification contradiction score.

3.4. Module C: The Consistency Engine (NLI-Based Analysis)

The Consistency Engine evaluates semantic consistency across the collected responses using a pre-trained Natural Language Inference model. It constructs a contradiction graph and derives graph-theoretic metrics that quantify the degree of agreement or disagreement among the model’s answers.

3.4.1. NLI Model

We employ cross-encoder/nli-deberta-v3-large **he2021debertav3**, a 435-million-parameter DeBERTa-v3 model fine-tuned on the Multi-Genre NLI (MultiNLI) corpus for three-class classification. Given a premise–hypothesis text pair, the model outputs a probability distribution over three labels:

$$\text{NLI}(R_i, R_j) = \{P_{\text{ent}}, P_{\text{neu}}, P_{\text{con}}\} \quad (3.6)$$

where P_{ent} , P_{neu} , and P_{con} denote the probabilities of entailment, neutral, and contradiction, respectively. The cross-encoder architecture processes the two texts jointly (separated by a [SEP] token), enabling rich cross-attention between them.

3.4.2. Pairwise Contradiction Matrix Construction

For each pair of responses (R_i, R_j) , we first extract the ANSWER portions using the `extract_answer` function:

$$A_i = \text{ExtractAnswer}(R_i), \quad A_j = \text{ExtractAnswer}(R_j) \quad (3.7)$$

NLI is inherently asymmetric: the relationship $\text{NLI}(A_i, A_j)$ may differ from $\text{NLI}(A_j, A_i)$. To account for this, we evaluate *both directions* and construct the contradiction matrix $\mathbf{M} \in \mathbb{R}^{n \times n}$ as follows. For each pair (i, j) with $i < j$:

$$(\ell_{ij}, c_{ij}) = \text{NLI}(A_i, A_j), \quad (\ell_{ji}, c_{ji}) = \text{NLI}(A_j, A_i) \quad (3.8)$$

where ℓ denotes the predicted label and c the associated confidence. The contradiction score is then computed based on the following rules:

$$\mathbf{M}[i][j] = \begin{cases} \max(c_{ij} \cdot \mathbb{I}[\ell_{ij} = \text{con}], c_{ji} \cdot \mathbb{I}[\ell_{ji} = \text{con}]) & \text{if } \ell_{ij} = \text{con} \vee \ell_{ji} = \text{con} \\ \frac{c_{ij} + c_{ji}}{2} \times 0.5 & \text{if } \ell_{ij} = \text{neu} \wedge \ell_{ji} = \text{neu} \\ c_{\text{neu}} \times 0.3 & \text{if exactly one is neutral} \\ 0 & \text{if both entailment} \end{cases} \quad (3.9)$$

The matrix is symmetric: $\mathbf{M}[j][i] = \mathbf{M}[i][j]$. The diagonal entries are zero by convention.

3.4.3. Graph-Based Consistency Metrics

From the contradiction matrix, we construct an undirected weighted graph $G = (V, E)$ where each response is a node and an edge is added between responses i and j if $\mathbf{M}[i][j] > \tau_{\text{edge}}$ (with $\tau_{\text{edge}} = 0.5551$ after optimization). Edge weights correspond to the contradiction scores. We then compute three graph-theoretic metrics:

1. **Graph Density (d):** The ratio of actual contradiction edges to the maximum possible number of edges.

$$d = \frac{|E|}{\binom{n}{2}} = \frac{2|E|}{n(n-1)} \quad (3.10)$$

A high density indicates widespread disagreement among responses.

2. **Average Clustering Coefficient ($\bar{c}$):** Measures the degree to which contradictions are locally clustered. Computed using the standard NetworkX implementation:

$$\bar{c} = \frac{1}{n} \sum_{v \in V} c_v \quad (3.11)$$

where c_v is the local clustering coefficient of node v . High clustering may indicate the presence of distinct “answer subgroups” that internally agree but mutually contradict.

3. **Fragmentation (f):** Measures how many disconnected components exist in the graph.

$$f = \frac{|\mathcal{C}| - 1}{n - 1} \quad (3.12)$$

where $|\mathcal{C}|$ is the number of connected components. A fragmented graph indicates multiple incompatible answer clusters.

3.4.4. Consistency Score

The overall consistency score combines the three graph metrics into a single value:

$$S_{\text{cons}} = 1 - (\alpha_g \cdot d + \beta_g \cdot \bar{m} + \gamma_g \cdot f) \quad (3.13)$$

where $\alpha_g = 0.4$, $\beta_g = 0.4$, $\gamma_g = 0.2$ are internal weights for density, average contradiction strength, and fragmentation, respectively. The average contradiction strength $\bar{m}$ is the mean of all off-diagonal entries:

$$\bar{m} = \frac{\sum_{i \neq j} \mathbf{M}[i][j]}{n(n-1)} \quad (3.14)$$

The consistency score $S_{\text{cons}} \in [0, 1]$, where values near 1 indicate high agreement and values near 0 indicate pervasive contradiction.

3.4.5. Self-Verification Score

The self-verification score quantifies the degree of contradiction between the model’s original answer A_0 and its self-verification response A_{verify} . It uses exactly the same bidirectional NLI scoring logic as the contradiction matrix (Equation 3.9):

$$S_{\text{sv}} = \text{ContradictionScore}(A_0, A_{\text{verify}}) \quad (3.15)$$

where $A_0 = \text{ExtractAnswer}(R_0)$ and $A_{\text{verify}} = \text{ExtractAnswer}(R_{\text{verify}})$. A high self-verification score indicates that the model, upon re-examination, contradicts its own initial answer—a strong hallucination signal.

3.5. Module D: The Complexity Engine (Information-Theoretic Analysis)

The Complexity Engine computes two information-theoretic metrics: token entropy (measuring the model’s internal uncertainty) and pairwise Normalized Compression Distance (measuring the informational dissimilarity between responses).

3.5.1. Token Entropy

Using the token-level log-probabilities $\{\log p(w_t \mid w_{<t})\}_{t=1}^T$ returned by the target LLM, we compute the average per-token entropy of each response:

$$H(R) = -\frac{1}{T} \sum_{t=1}^T \log p(w_t \mid w_{<t}) \quad (3.16)$$

where T is the number of tokens in the response and $p(w_t \mid w_{<t})$ is the conditional probability assigned by the model to token w_t given all preceding tokens. This quantity, measured in nats (natural log) or bits (log base 2), captures the model’s average uncertainty during generation: a response generated with consistently high confidence (low entropy) is more likely to reflect genuine knowledge, while high entropy may indicate guessing or uncertainty.

3.5.2. Entropy Normalization

To map the raw entropy value to the $[0, 1]$ range for fusion, we apply min-max normalization with an empirically calibrated upper bound:

$$H_{\text{norm}} = \min\left(\frac{H(R)}{H_{\text{max}}}, 1.0\right) \quad (3.17)$$

where H_{max} is an upper-bound calibration parameter. Through Bayesian optimization, H_{max} was tuned to approximately 6.29 for GPT-4o-mini responses on TruthfulQA-style questions.

3.5.3. Pairwise Normalized Compression Distance (NCD)

Normalized Compression Distance (NCD) [8], [9] is an approximation of the uncomputable Normalized Information Distance based on Kolmogorov complexity. For two texts x and y , NCD is defined as:

$$\text{NCD}(x, y) = \frac{C(xy) - \min(C(x), C(y))}{\max(C(x), C(y))} \quad (3.18)$$

where $C(\cdot)$ denotes the compressed size (in bytes) using the zlib compression algorithm, and xy denotes the concatenation of texts x and y . NCD values range from approximately 0 (identical or highly similar) to approximately 1 (maximally dissimilar).

Critically, the NCD signal used in LOGIC-HALT is the *average pairwise NCD* computed across all pairs of responses, *not* the compression ratio of individual responses. Given n responses, we compute:

$$\overline{\text{NCD}} = \frac{2}{n(n-1)} \sum_{i=1}^n \sum_{j=i+1}^n \text{NCD}(R_i, R_j) \quad (3.19)$$

A high average pairwise NCD indicates that the responses are informationally dissimilar to one another, suggesting that the model is producing divergent content for semantically equivalent questions—a hallucination signal.

3.5.4. NCD Normalization

The average pairwise NCD is normalized to $[0, 1]$ using calibrated bounds:

$$\text{NCD}_{\text{norm}} = \min\left(\frac{\overline{\text{NCD}}}{\text{NCD}_{\text{max}}}, 1.0\right) \quad (3.20)$$

where $\text{NCD}_{\text{max}} \approx 0.7978$ is the optimized upper bound.

3.6. Module F: The Answer Validator (Answer-Level Consistency)

While the Consistency Engine operates at the semantic level (comparing the meaning of answer texts via NLI), the Answer Validator operates at the *answer extraction level*, directly comparing the final answers given by the model. This module provides a complementary signal that addresses a limitation of NLI-based analysis: two responses may have similar semantic structure but contain different factual conclusions.

3.6.1. Answer Extraction

The Answer Validator uses regex-based pattern matching to extract the final answer from each response. The extraction pipeline searches for several patterns in order of specificity:

1. **Numeric answers:** Patterns matching “the answer is X”, “result is X”, “= X”, and final numbers.
2. **Boolean/categorical answers:** Patterns matching “yes”, “no”, “true”, “false”, and their Turkish equivalents (“evet”, “hayır”).
3. **Named entities:** Key phrases extracted from the ANSWER portion.
4. **Fallback:** The last number appearing in the response text.

3.6.2. Majority Voting

Given the extracted answers $\{a_1, a_2, \dots, a_n\}$ from all n responses, we perform majority voting:

$$a^* = \arg \max_{a \in \mathcal{A}} \sum_{i=1}^n \mathbb{1}[a_i = a] \quad (3.21)$$

where $\mathcal{A}$ is the set of unique extracted answers. The majority ratio is:

$$r_{\text{maj}} = \frac{|\{i : a_i = a^*\}|}{n} \quad (3.22)$$

3.6.3. Minority Penalty

Responses whose extracted answer disagrees with the majority receive a *minority penalty*. The aggregated minority penalty for the question is determined by the strength of the majority consensus:

$$P_{\text{minority}} = \begin{cases} 0.00 & \text{if } r_{\text{maj}} \geq 0.80 \\ 0.10 & \text{if } 0.60 \leq r_{\text{maj}} < 0.80 \\ 0.30 & \text{if } r_{\text{maj}} < 0.60 \end{cases} \quad (3.23)$$

The minority penalty proved to be the single most important detection signal in the final optimized system, receiving the highest fusion weight ($w_5 = 0.3919$). This result is intuitive: if a model gives different final answers to semantically equivalent questions, that is the most direct evidence of unreliable reasoning, regardless of how consistent or confident the explanations may appear.

3.7. Module E: The Fusion Layer (Five-Signal Decision)

The Fusion Layer aggregates all signals into a single hallucination risk score. We employ a linear weighted combination of five normalized signals, where the weights are optimized via Bayesian hyperparameter search.

3.7.1. Five-Signal Fusion Formula

The hallucination risk is computed as:

$$\text{Risk} = w_1 \cdot S_{\text{sv}} + w_2 \cdot (1 - S_{\text{cons}}) + w_3 \cdot H_{\text{norm}} + w_4 \cdot \text{NCD}_{\text{norm}} + w_5 \cdot P_{\text{minority}} \quad (3.24)$$

where:

- S_{sv} is the self-verification contradiction score (Section 3.4.5)
- $(1 - S_{\text{cons}})$ is the inconsistency score (inverted consistency, higher = more inconsistent)
- H_{norm} is the normalized token entropy (Equation 3.17)
- NCD_{norm} is the normalized pairwise NCD
- P_{minority} is the minority penalty from the Answer Validator (Equation 3.23)

All five input signals are in the range $[0, 1]$, where higher values indicate greater hallucination suspicion. The weights are constrained to sum to unity: $\sum_{i=1}^5 w_i = 1.0$.

3.7.2. Bayesian Weight Optimization

The fusion weights were optimized using the Tree-structured Parzen Estimator (TPE) algorithm **bergstra2011algorithms** implemented in the Optuna framework **akiba2019optuna**. The optimization procedure was as follows:

- **Dataset:** 817 questions from the TruthfulQA benchmark **lin2022truthfulqa**
- **Labels:** Determined via NLI entailment between the LLM’s answer and the ground-truth answer, using a *separate* NLI model (**microsoft/deberta-large-mnli**) to avoid data leakage with the consistency features
- **Search space:** Continuous intervals for each weight, threshold, and normalization parameter

- **Objective:** Maximize F1 score
- **Trials:** 500 Optuna trials with 15 random startup trials
- **Hardware:** NVIDIA RTX 5070 (12GB VRAM), with FP16 inference for the NLI model

The optimization procedure pre-computes all NLI pair scores, entropy values, pairwise NCD, self-verification scores, and minority penalties once on the GPU, then evaluates different weight combinations purely in CPU memory. This allows 500 trials to complete in approximately 30 minutes.

3.7.3. Optimized Parameters

Table 3.1 presents the final optimized weights and threshold.

Table 3.1: Bayesian-optimized fusion weights and threshold (500 TPE trials, 817 TruthfulQA questions). All weights are normalized to sum to 1.0.

Signal	Symbol	Weight	Contribution
Self-Verification	w_1	0.0852	8.52%
Inconsistency	w_2	0.2120	21.20%
Token Entropy	w_3	0.1493	14.93%
Pairwise NCD	w_4	0.1616	16.16%
Minority Penalty	w_5	0.3919	39.19%
Decision Threshold τ		0.2848	
Best F1 Score		0.7346	

Several observations about the optimized weights merit discussion:

1. **Minority Penalty dominates (39.19%):** The answer extraction and majority voting signal carries the highest weight. This indicates that at the answer level, inconsistency across semantically equivalent questions is the most reliable hallucination indicator. When a model gives different final answers to the same question rephrased in different ways, this is the strongest evidence of unreliable reasoning.
2. **Inconsistency is second (21.20%):** The NLI-based semantic inconsistency signal provides substantial discriminative power. It captures cases where the model produces contradictory explanations even when the final answers might coincidentally agree.

3. **NCD and Entropy contribute moderately (16.16% and 14.93%):** The information-theoretic signals provide complementary evidence. High NCD indicates divergent response content; high entropy indicates internal uncertainty during generation.
4. **Self-Verification is smallest (8.52%):** While useful, the self-verification signal is the weakest contributor. This is understandable: when asked to fact-check itself, a hallucinating model may simply re-affirm its incorrect answer with high confidence, limiting the discriminative power of this signal.
5. **Low threshold ($\tau = 0.2848$):** The relatively low threshold reflects the system’s design bias toward high recall—it is calibrated to catch as many hallucinations as possible, even at the cost of some false positives.

3.7.4. Classification Decision

The final binary classification is:

$$\text{Decision} = \begin{cases} \text{HALLUCINATION} & \text{if Risk} \geq \tau \\ \text{SAFE} & \text{if Risk} < \tau \end{cases} \quad (3.25)$$

where $\tau = 0.2848$ is the optimized decision threshold.

3.7.5. Confidence Estimation

In addition to the binary decision, the system reports a confidence level based on the distance between the risk score and the threshold:

$$\text{Confidence} = \begin{cases} \text{High} & \text{if } |\text{Risk} - \tau| \geq 0.70 - \tau \\ \text{Medium} & \text{if } \tau - 0.35 \leq |\text{Risk} - \tau| < 0.70 - \tau \\ \text{Low} & \text{otherwise} \end{cases} \quad (3.26)$$

This allows downstream consumers of the detection result to modulate their response based on how certain the system is about its classification.

3.8. Complete Algorithm

The complete LOGIC-HALT detection procedure is presented in Figure 3.2. The algorithm takes as input a question Q , a target LLM $\mathcal{M}$, and the set of optimized

parameters, and outputs a hallucination risk score with a binary classification.

3.9. Computational Complexity Analysis

The computational cost of the LOGIC-HALT pipeline is dominated by two operations: (1) LLM API calls in Modules A and B, and (2) NLI inference in Module C.

- **LLM API calls:** For each input question, the system makes k calls to the teacher LLM (variant generation), $k + 1$ calls to the target LLM (response collection), and 1 call for self-verification, totaling $2k + 2$ API calls. With $k = 5$, this is 12 API calls per question.
- **NLI inference:** The Consistency Engine evaluates $\binom{n}{2}$ pairs in *both* directions, yielding $n(n - 1)$ NLI inferences. With $n = 6$ responses, this is 30 NLI forward passes. Additionally, self-verification requires 2 NLI passes (bidirectional). On an NVIDIA RTX 5070 with FP16 inference and batch size 64, all 32 NLI inferences complete in under 1 second.
- **Pairwise NCD:** The $\binom{n}{2} = 15$ pairwise NCD computations involve zlib compression of text pairs, completing in negligible time (sub-millisecond per pair).
- **Answer extraction and majority voting:** Regex-based extraction and counting are $O(n)$ operations.

The overall per-question latency is dominated by the LLM API calls (typically 1–3 seconds each), with all local computation completing in approximately 1–2 seconds on GPU hardware.

3.10. Summary

This chapter presented the complete LOGIC-HALT methodology for inference-time hallucination detection. The key contributions of the framework are:

1. **Metamorphic variant generation** using five transformation operators with semantic filtering via Sentence-BERT (all-MiniLM-L6-v2, $\tau_{\text{sim}} = 0.75$).
2. **Hybrid response format** separating ANSWER from REASONING, enabling NLI comparison on answer portions only to reduce false positives from explanation style variance.

Algorithm: LOGIC-HALT Hallucination Detection

Input: Question Q , Target LLM $\mathcal{M}$, Teacher LLM $\mathcal{T}$, Weights $\{w_1, \dots, w_5\}$, Threshold τ

Output: Hallucination risk score $\in [0, 1]$ and binary classification

// Phase 1: Variant Generation (Module A)

1. **for each** $T_j \in \{T_1, T_2, T_3, T_4, T_5\}$ **do**
2. $Q_j \leftarrow \mathcal{T}(T_j, Q)$ *// Generate variant via teacher LLM*
3. $\text{sim}_j \leftarrow \text{CosineSim}(\text{SBERT}(Q), \text{SBERT}(Q_j))$
4. **if** $\text{sim}_j < 0.75$ **then discard** Q_j

// Phase 2: Response Collection (Module B)

5. **for each** $Q_i \in \{Q, Q_1, \dots, Q_k\}$ **do**
6. $R_i \leftarrow \mathcal{M}(Q_i)$ with temperature = 0.3, logprobs = true
7. $R_{\text{verify}} \leftarrow \mathcal{M}(\text{FactCheckPrompt}(Q, R_0.\text{answer}))$ with temperature = 0.0

// Phase 3: Self-Verification Score (Module C)

8. $S_{\text{sv}} \leftarrow \text{NLI_Contradiction}(R_0.\text{answer}, R_{\text{verify}}.\text{answer})$

// Phase 4: Consistency Analysis (Module C)

9. **for each** pair (R_i, R_j) with $i < j$ **do**
10. $M[i][j] \leftarrow \text{BiNLI}(R_i.\text{answer}, R_j.\text{answer})$ *// Bidirectional NLI*
11. Build graph G from M with edge threshold τ_{edge}
12. $S_{\text{cons}} \leftarrow 1 - (\alpha_g \cdot \text{density}(G) + \beta_g \cdot \text{mean}(M) + \gamma_g \cdot \text{frag}(G))$

// Phase 5: Complexity Analysis (Module D)

13. $H_{\text{norm}} \leftarrow \text{NormalizeEntropy}\left(-\frac{1}{T} \sum_t \log p(w_t \mid w_{<t})\right)$
14. $\text{NCD}_{\text{norm}} \leftarrow \text{Normalize}\left(\frac{2}{n(n-1)} \sum_{i < j} \text{NCD}(R_i, R_j)\right)$

// Phase 6: Answer Validation (Module F)

15. $\{a_1, \dots, a_n\} \leftarrow \text{ExtractAnswers}(\{R_0, R_1, \dots, R_k\})$
16. $a^* \leftarrow \text{MajorityVote}(\{a_1, \dots, a_n\})$
17. $P_{\text{minority}} \leftarrow \text{MinorityPenalty}(r_{\text{maj}})$ *// Based on majority ratio*

// Phase 7: Fusion (Module E)

18. $\text{Risk} \leftarrow w_1 S_{\text{sv}} + w_2 (1 - S_{\text{cons}}) + w_3 H_{\text{norm}} + w_4 \text{NCD}_{\text{norm}} + w_5 P_{\text{minority}}$
19. **if** $\text{Risk} \geq \tau$ **then return** $\{\text{Risk}, \text{HALLUCINATION}\}$
20. **else return** $\{\text{Risk}, \text{SAFE}\}$

Figure 3.2: The LOGIC-HALT hallucination detection algorithm. The pipeline proceeds through seven phases: variant generation, response collection, self-verification, consistency analysis, complexity analysis, answer validation, and five-signal fusion.

3. **Bidirectional NLI consistency analysis** using cross-encoder/nli-deberta-v3-large (435M parameters), with graph-based metrics (density, clustering, fragmentation).
4. **Self-verification** as a complementary signal, using NLI to detect contradictions between the model’s original answer and its own fact-check.
5. **Pairwise NCD** as the correct information-theoretic dissimilarity measure between responses (not single-text compression ratios).
6. **Answer-level majority voting** with minority penalty, which emerged as the strongest individual signal (39.19% of fusion weight).
7. **Five-signal Bayesian-optimized fusion** that operates entirely at inference time without requiring ground-truth answers, achieving $F1 = 0.7346$ on TruthfulQA with 817 questions.
8. **Honest evaluation**: explicit acknowledgment that the GT-dependent design ($F1 = 0.7730$) was unusable at inference time, motivating the redesign to inference-time-only signals.

The next chapter describes the experimental setup, implementation details, and the benchmarks used for evaluation.

4. EXPERIMENTAL SETUP

This chapter presents the implementation details, hardware environment, dataset selection, evaluation methodology, and experimental procedures employed to evaluate the LOGIC-HALT hallucination detection system. All configuration parameters, software versions, and optimization settings reported herein correspond to the actual deployed system.

4.1. Implementation

4.1.1. Technology Stack

The LOGIC-HALT system is implemented in Python 3.10+ and relies on a curated set of libraries spanning deep learning, natural language inference, Bayesian optimization, and web-based demonstration. Table 4.1 lists every major dependency together with its minimum version and role within the pipeline.

4.1.2. System Architecture

The implementation follows a modular, pipeline-oriented design. Each stage of the LOGIC-HALT architecture described in Chapter 3 maps to a dedicated Python module:

Additional scripts support the experimental workflow:

- `scripts/batch_api_helper.py` — Prepares and manages OpenAI Batch API requests for cost-efficient, large-scale data collection.
- `scripts/batch_optimization.py` — Implements GPU-accelerated Bayesian optimization over pre-computed features.
- `scripts/ablation_study.py` — Runs a systematic ablation study across $2^3 = 8$ signal combinations.
- `scripts/generate_visualizations.py` — Produces publication-quality figures and confusion matrices.

All configuration is centralized in two YAML files: `config/config.yaml` for hyperparameters and hardware settings, and `config/prompts.yaml` for all LLM prompt templates.

Table 4.1: Software dependencies and minimum versions.

Library	Version	Purpose
PyTorch	$\geq 2.1.0$	Deep learning framework with CUDA 12.8 support
Transformers	$\geq 4.35.0$	Hugging Face model loading for NLI cross-encoders
Sentence-Transformers	$\geq 2.2.2$	Sentence-BERT embeddings for semantic similarity filtering and ablation
Optuna	$\geq 3.4.0$	Bayesian hyperparameter optimization (TPE sampler)
OpenAI API	$\geq 1.3.0$	GPT-4o-mini access for response generation and variant synthesis
Flask	$\geq 2.3.0$	Web-based demonstration interface
NetworkX	$\geq 3.2.0$	Contradiction graph construction and analysis
scikit-learn	$\geq 1.3.0$	Classification metrics and cross-validation utilities
NumPy	$\geq 1.24.3$	Numerical computations
pandas	$\geq 2.0.3$	Tabular data handling for optimization trial logs
Matplotlib / Seaborn	$\geq 3.7 / 0.12$	Visualization of heatmaps, graphs, and gauges
PyYAML	$\geq 6.0.1$	Configuration file parsing
python-dotenv	$\geq 1.0.0$	Environment variable management for API keys
tqdm	$\geq 4.66.1$	Progress bars for batch inference and optimization

Table 4.2: Source modules and their architectural roles.

Module	Source File	Function
A – Morpher	src/morpher.py	Metamorphic variant generation via LLM
B – Interrogator	src/interrogator.py	LLM response collection with log-probability
C – Consistency Engine	src/consistency.py	Pairwise NLI and contradiction graph analysis
D – Complexity Engine	src/complexity.py	Entropy and Normalized Compression Distance
E – Fusion Layer	src/fusion.py	Five-signal weighted decision mechanism
F – Answer Validator	src/answer_validator.py	Answer extraction and majority voting
Pipeline Orchestrator	src/detector.py	End-to-end integration of Modules A–F
Web Interface	web/app.py	Flask-based real-time demonstration

4.1.3. NLI Model Selection

For pairwise consistency analysis (Module C), we employ the `cross-encoder/nli-deberta-v3-large` model from the Hugging Face Model Hub. This is a 435-million-parameter cross-encoder fine-tuned on the Multi-Genre Natural Language Inference (MultiNLI) corpus. The model accepts a premise–hypothesis pair and outputs a probability distribution over three labels:

$$P(\text{label} \mid \text{premise}, \text{hypothesis}) \in \{\text{contradiction}, \text{neutral}, \text{entailment}\} \quad (4.1)$$

with the label mapping 0 = contradiction, 1 = neutral, 2 = entailment.

The *large* variant was chosen over the *base* variant (184M parameters) because the larger model provides substantially higher accuracy on the MultiNLI benchmark, and the 12 GB VRAM of our GPU accommodates it comfortably with FP16 inference. Table 4.3 summarizes the NLI model configuration.

Table 4.3: NLI model configuration.

Parameter	Value
Model identifier	<code>cross-encoder/nli-deberta-v3-large</code>
Parameters	435M
Inference precision	FP16 (half-precision on GPU)
Batch size	64
Maximum sequence length	512 tokens
Label mapping	0 = contradiction, 1 = neutral, 2 = entailment

A critical design decision concerns the *hybrid comparison format*. The NLI model compares only the extracted ANSWER portion of each LLM response, not the full text including reasoning. This prevents false positive contradictions that arise merely from different explanation styles, while preserving sensitivity to genuine factual disagreements.

4.1.4. Labeling Model

To compute ground-truth labels for the TruthfulQA dataset, a *separate* NLI model is used: `microsoft/deberta-large-mnli`. This deliberate separation avoids data leakage—the model used for labeling is entirely distinct from the model used for feature extraction. The labeling procedure is as follows:

1. For each question, the LLM’s extracted answer is compared against the dataset’s reference ground truth using the labeling NLI model.
2. The model outputs probabilities for *entailment*, *neutral*, and *contradiction*.
3. The label is assigned according to the following rule:
 - If $P(\text{entailment}) > 0.5$: label = 0 (truthful).
 - If $P(\text{contradiction}) > 0.3$: label = 1 (hallucination).
 - If $P(\text{entailment}) < 0.3$: label = 1 (hallucination).
 - Otherwise: label = 0 (default to truthful when uncertain).

This NLI-based labeling scheme is substantially more robust than string matching or cosine similarity, as it accounts for semantic equivalence between differently-worded correct answers.

4.1.5. Target LLM

We evaluate the LOGIC-HALT system using OpenAI’s GPT-4o-mini as the target LLM. This model was selected for the following reasons:

1. **Log-probability access:** The OpenAI API exposes per-token log-probabilities (top-5), which are essential for the entropy signal in Module D.
2. **Consistent API access:** Deterministic querying with fixed temperature settings yields reproducible results.
3. **Representativeness:** GPT-4o-mini represents the capabilities and failure modes of modern instruction-tuned LLMs while being cost-effective for large-scale experimentation.
4. **Batch API support:** The OpenAI Batch API enables 50% cost reduction for large-scale data collection, processing requests within a 24-hour window.

GPT-4o-mini serves a dual role: it generates both the original responses (as the LLM under test) and the semantic variants (as the Morpher backbone). For self-verification, a stronger model (GPT-4o) is used at temperature 0.0 to provide high-quality fact-checking.

Table 4.4 lists the response generation parameters.

The structured response format (ANSWER followed by REASONING) is critical to the system’s accuracy. By separating the factual claim from its justification, the NLI engine can compare answers directly without being confounded by explanation-style variance.

Table 4.4: LLM interrogation parameters.

Parameter	Response Generation	Variant Generation
Model	GPT-4o-mini	GPT-4o-mini
Temperature	0.6	0.7
Maximum tokens	400	500
Log-probabilities	Enabled (top 5)	—
Samples per query	1	1 per transformation
Response format	ANSWER + REASONING	JSON {variant, type}

4.2. Datasets

4.2.1. Primary Dataset: TruthfulQA

The primary evaluation dataset is **TruthfulQA** [lin2022truthfulqa](#), a standard benchmark specifically designed to measure whether language models generate truthful responses. The dataset comprises **817 questions** across multiple categories that are deliberately crafted to elicit common misconceptions, false beliefs, and factual errors from LLMs. Table 4.5 provides an overview.

Table 4.5: TruthfulQA dataset overview.

Property	Value
Total questions	817
Question categories	Health, Law, Finance, Politics, History, Science, etc.
Design principle	Questions that elicit false beliefs and misconceptions
Reference answers	Ground truth and known wrong answers per question
Evaluation standard	Widely used in hallucination detection literature

Each question in TruthfulQA includes a reference ground truth answer and a set of known incorrect answers. This structure makes it particularly suitable for our evaluation because:

1. The questions are designed to probe specific failure modes of LLMs, such as repeating popular misconceptions.
2. The ground truth answers enable automated label computation via NLI entailment (Section 4.1.4).
3. The dataset is widely adopted in the hallucination detection literature, facilitating comparison with prior work.

4.2.2. Pilot Dataset

During initial development, a custom pilot dataset of **20 questions** was used for rapid prototyping and debugging. This dataset spans ten categories (factual, mathematical, scientific, person, quote, mixed, logical, temporal, technical, historical) and was designed with known SAFE and hallucination-prone (HAL) questions. Table 4.6 shows the composition.

Table 4.6: Pilot dataset composition by category.

Category	Count	Expected SAFE	Expected HAL
Factual	4	2	2
Mathematical	3	2	1
Scientific	2	1	1
Person	2	1	1
Quote	2	1	1
Mixed	2	0	2
Logical	1	1	0
Temporal	1	0	1
Technical	2	1	1
Historical	1	0	1
Total	20	9	11

The pilot dataset served exclusively for system integration testing and was not used in the final evaluation. All results reported in Chapter 5 are based solely on the TruthfulQA benchmark.

4.2.3. Data Collection via Batch API

Responses from GPT-4o-mini were collected using the **OpenAI Batch API**, which processes requests asynchronously within a 24-hour window at 50% reduced cost. The data collection pipeline proceeds in three stages:

1. **Variant generation:** For each of the 817 TruthfulQA questions, five semantic variants are generated using the transformations defined in Section 3.3.1 (paraphrase, negation, variable substitution, premise reordering, redundant context). This yields $817 \times 5 = 4,085$ variant generation requests.
2. **Original response collection:** Each original question is sent to GPT-4o-mini with the structured ANSWER + REASONING prompt format. This produces 817 original responses.

3. **Variant response collection:** Each generated variant is independently queried, producing up to $817 \times 5 = 4,085$ variant responses.

The Batch API helper (`scripts/batch_api_helper.py`) automates the preparation of JSONL request files, submission, status monitoring, and result parsing. All responses are organized into a structured JSON format keyed by question ID and variant type.

4.3. Evaluation Metrics

4.3.1. Classification Metrics

Hallucination detection is formulated as a binary classification task where the positive class represents a hallucination. We evaluate the system using four standard metrics:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (4.2)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (4.3)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (4.4)$$

$$\text{F1-Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4.5)$$

where the confusion matrix elements are defined as:

- TP (True Positive): Hallucination correctly detected.
- TN (True Negative): Truthful response correctly classified as safe.
- FP (False Positive): Truthful response incorrectly flagged as hallucination.
- FN (False Negative): Hallucination missed by the system.

4.3.2. Confusion Matrix

The confusion matrix provides granular insight into the classifier’s error distribution:

Table 4.7: Confusion matrix structure for hallucination detection.

		Predicted	
		SAFE	HAL
Actual	SAFE	TN	FP
	HAL	FN	TP

In the context of hallucination detection, **false negatives** (missed hallucinations) are particularly dangerous in safety-critical applications, while **false positives** (unnecessary flags on truthful content) degrade user trust. The F1-score provides a balanced view, and we additionally report precision and recall separately to characterize the trade-off.

4.3.3. Optimization Objective

The Bayesian optimization procedure uses a **precision-balanced F1 objective** rather than raw F1. Specifically, if precision falls below a minimum threshold of 0.60, the objective value is penalized:

$$\text{Objective} = \begin{cases} F_1 \cdot \frac{P}{0.60} & \text{if } P < 0.60 \\ F_1 \cdot (1 + 0.15 \cdot \min(P - 0.60, 0.35)) & \text{otherwise} \end{cases} \quad (4.6)$$

where P denotes precision and F_1 denotes the F1-score. This objective encourages the optimizer to find configurations that maintain acceptable precision while maximizing overall detection performance.

4.4. Experimental Configuration

4.4.1. Five-Signal Fusion Parameters

The fusion layer (Module E) combines five signals to produce a scalar hallucination risk score. Each signal is normalized to the $[0, 1]$ range where higher values indicate greater suspicion of hallucination:

$$\text{Risk} = w_1 \cdot S_{sv} + w_2 \cdot S_{incon} + w_3 \cdot S_{ent} + w_4 \cdot S_{ncd} + w_5 \cdot S_{mp} \quad (4.7)$$

where the five signals and their optimized weights are listed in Table 4.8.

The dominance of the minority penalty weight ($w_5 = 0.3919$) is noteworthy: it

Table 4.8: Optimized five-signal fusion weights (500 trials, 817 questions).

Symbol	Signal	Weight	Description
w_1	Self-Verification (S_{sv})	0.0852	NLI contradiction between original and variant answers
w_2	Inconsistency (S_{incon})	0.2120	Self-consistency: $1 - \text{consistency score}$
w_3	Entropy (S_{ent})	0.1493	Normalized average token entropy
w_4	NCD (S_{ncd})	0.1616	Pairwise Normalized Compression Distance
w_5	Minority Penalty (S_{mp})	0.3919	Majority voting disagreement penalty
τ	Hallucination threshold	0.2848	Decision boundary for binary classification

indicates that answer-level disagreement among variants is the single strongest predictor of hallucination, consistent with the hypothesis that factually grounded answers exhibit higher inter-variant stability.

4.4.2. NLI Consistency Parameters

The consistency engine applies learned thresholds to the NLI probability outputs:

Table 4.9: NLI consistency engine parameters (Bayesian-optimized).

Parameter	Symbol	Value
Contradiction weight	w_c	0.6353
Neutral weight	w_n	0.6432
Entailment weight	w_e	0.0
Minimum edge weight	θ_{edge}	0.5551

For each pair of responses (r_i, r_j) , a combined contradiction score is computed as:

$$s_{ij} = w_c \cdot P(\text{contradiction} \mid r_i, r_j) + w_n \cdot P(\text{neutral} \mid r_i, r_j) \quad (4.8)$$

An edge is added to the contradiction graph whenever $s_{ij} > \theta_{\text{edge}}$. The overall consistency score is then derived from the graph density, average contradiction strength, and fragmentation (number of connected components).

4.4.3. Complexity Engine Parameters

Table 4.10 lists the parameters governing the entropy and NCD computations.

Entropy is calculated from per-token log-probabilities provided by the OpenAI

Table 4.10: Complexity engine parameters.

Parameter	Value
Compression algorithm	zlib
Entropy method	Token-level ($-\frac{1}{T} \sum_t \log p(x_t)$)
Entropy normalization max	6.29 bits
NCD normalization max	0.7978
NCD computation	Pairwise across all response pairs

API. The pairwise NCD is computed using the standard formula:

$$\text{NCD}(x, y) = \frac{C(xy) - \min(C(x), C(y))}{\max(C(x), C(y))} \quad (4.9)$$

where $C(\cdot)$ denotes the compressed length using zlib. The average pairwise NCD across all response pairs for a given question is used as the fusion signal.

4.4.4. Answer Validator Parameters

Table 4.11: Answer validator parameters.

Parameter	Value
Minority penalty	0.05
Minimum responses	3
High confidence threshold	80% majority agreement
Low confidence threshold	50% majority agreement

The answer validator extracts the core factual answer from each structured response using regular expression patterns and then performs majority voting. Responses whose extracted answer disagrees with the majority receive a penalty that is incorporated into the minority penalty signal (S_{mp}).

4.4.5. Morpher Transformation Parameters

Five transformation types are applied to each question: *paraphrase*, *negation* (double negation or contrapositive), *variable substitution*, *premise reordering*, and *redundant context injection*. A variant is retained only if its cosine similarity to the original question exceeds 0.75, ensuring semantic equivalence while encouraging syntactic diversity.

Table 4.12: Morpher (Module A) configuration.

Parameter	Value
Number of variants per question	5
Generation model	GPT-4o-mini
Temperature	0.7
Maximum tokens	500
Similarity threshold	0.75 (cosine, Sentence-BERT)
Similarity model	all-MiniLM-L6-v2

4.5. Bayesian Optimization

4.5.1. Optimization Framework

Hyperparameter optimization is performed using Optuna **akiba2019optuna** with the Tree-structured Parzen Estimator (TPE) sampler. The optimization search space encompasses the five fusion weights, the decision threshold, NLI scoring parameters, and normalization bounds—13 hyperparameters in total. Table 4.13 summarizes the optimization configuration.

Table 4.13: Bayesian optimization configuration.

Parameter	Value
Framework	Optuna 3.4+
Sampler	TPE (Tree-structured Parzen Estimator)
Number of trials	500
Startup trials (random)	15
Objective	Precision-balanced F1 (Equation 4.6)
Dataset	TruthfulQA (817 questions)

4.5.2. Search Space

Table 4.14 defines the search ranges for all optimized hyperparameters.

The minority penalty weight ϵ is not independently sampled; it is derived as the residual $\epsilon = 1 - \alpha - \beta - \gamma - \delta$, ensuring that the five weights always sum to unity. Trials where $\epsilon < 0$ or $\epsilon > 0.40$ are pruned.

Table 4.14: Hyperparameter search space.

Parameter	Role	Min	Max
α (self-verification weight)	Fusion weight	0.05	0.50
β (inconsistency weight)	Fusion weight	0.05	0.40
γ (entropy weight)	Fusion weight	0.05	0.30
δ (NCD weight)	Fusion weight	0.01	0.30
ϵ (minority penalty weight)	Derived: $1 - \alpha - \beta - \gamma - \delta$	0.00	0.40
Hallucination threshold	Decision boundary	0.20	0.70
min_edge_weight	NLI graph threshold	0.15	0.60
contradiction_weight (w_c)	NLI scoring	0.50	1.00
neutral_weight (w_n)	NLI scoring	0.20	0.70
entropy_max	Normalization bound	5.0	15.0
ncd_max	Normalization bound	0.50	1.00

4.5.3. Pre-Computation Strategy

A key engineering contribution is the **pre-computation strategy** that decouples GPU-intensive NLI inference from the optimization loop. Before the first trial begins, the following quantities are computed once and cached in memory:

1. **NLI pair scores:** For every pair of responses within each question, the NLI model produces contradiction and neutral probabilities. These raw probabilities are stored, and each trial applies its own threshold and weighting parameters to derive the consistency score.
2. **Self-verification scores:** NLI contradiction probabilities between the original response and each variant response.
3. **Minority penalties:** Answer extraction and majority voting results for all 817 questions.
4. **Pairwise NCD:** Average pairwise Normalized Compression Distance for all response sets.
5. **Token entropy:** Per-question average token entropy.

This strategy reduces each optimization trial to a simple arithmetic operation over cached values, achieving approximately **0.01 seconds per trial**. The entire 500-trial optimization thus completes in under one minute of wall-clock time (excluding the one-time pre-computation phase).

4.6. Hardware Environment

All experiments were conducted on a single workstation whose specifications are listed in Table 4.15.

Table 4.15: Hardware configuration.

Component	Specification
GPU	NVIDIA GeForce RTX 5070 (12 GB GDDR7 VRAM)
CPU	AMD Ryzen 5 7500F (6 cores, 12 threads)
RAM	32 GB DDR5
CUDA	12.8
OS	Windows 11 Pro

The following CUDA optimizations are enabled to maximize GPU throughput:

- `torch.backends.cudnn.benchmark = True` — Automatic convolution algorithm tuning.
- `torch.backends.cuda.matmul.allow_tf32 = True` — TF32 tensor core acceleration for matrix multiplications.
- `torch.backends.cudnn.allow_tf32 = True` — TF32 for cuDNN operations.
- **FP16 mixed precision:** The NLI model weights are converted to half-precision (`model.half()`) for inference, halving memory consumption and approximately doubling throughput.

With these optimizations, the 435M-parameter NLI model operates comfortably within the 12 GB VRAM budget at a batch size of 64. The estimated GPU memory usage during batch NLI inference is approximately $2 + 64 \times 0.05 \approx 5.2$ GB.

4.7. Ablation Study Design

To quantify the contribution of three newly proposed signals, a systematic ablation study is conducted. The three signals under investigation are:

1. **Negation Contradiction (Neg):** Detects whether the LLM agrees with both a claim and its logical negation. If the NLI model finds high entailment between the original response and the negation-variant response, the model is exhibiting logical inconsistency.

2. **Factual Token Variance (FTV):** Measures the stability of factual content (numbers, proper nouns) across variant responses using Jaccard distance. High variance in factual tokens suggests confabulation.
3. **Semantic Answer Matching (SAM):** Replaces exact string matching in majority voting with Sentence-BERT cosine similarity. Answers with low average semantic similarity to other answers are identified as outliers.

The ablation enumerates all $2^3 = 8$ combinations, including the baseline (no new signals) and the full model (all three). Each configuration undergoes independent Bayesian optimization with 500 trials and 30 startup trials, using the precision-balanced F1 objective (Equation 4.6).

Table 4.16: Ablation study signal combinations.

Configuration	Neg	FTV	SAM	Total Signals
Baseline	—	—	—	5
Neg	✓	—	—	6
FTV	—	✓	—	6
SAM	—	—	✓	6
Neg+FTV	✓	✓	—	7
Neg+SAM	✓	—	✓	7
FTV+SAM	—	✓	✓	7
Neg+FTV+SAM	✓	✓	✓	8

4.8. Web Interface

A Flask-based web interface provides real-time demonstration of the LOGIC-HALT system, accessible at <http://127.0.0.1:5000>. The interface supports two modes of operation:

1. **Multi-Response Analysis Mode:** The user enters a question; the system generates 10 responses from GPT-4o-mini (all at temperature 0.3), performs the full LOGIC-HALT analysis pipeline (consistency, entropy, NCD, majority voting, fusion), and displays per-response risk scores with explanations.
2. **Question–Answer Pair Verification Mode:** The user provides both a question and a candidate answer. The system generates three variant responses at different temperatures (0.3, 0.5, 0.7), compares the user’s answer against these variants using NLI consistency analysis, and renders a hallucination verdict.

Both modes employ a **two-phase detection** strategy:

- **Phase 1 (Fast Check):** Extract answers from all responses and perform majority voting. If $\geq 80\%$ of responses agree, the question is fast-tracked to a “safe” classification without full NLI analysis.
- **Phase 2 (Detailed Analysis):** If Phase 1 detects disagreement, the full pipeline is invoked—NLI contradiction matrix construction, complexity analysis, and five-signal fusion.

The web interface also exposes a `/health` endpoint for monitoring, a `/statistics` endpoint for aggregate performance metrics, and a `/save_labels` endpoint for collecting human annotations.

4.9. Experimental Procedure

The complete evaluation procedure consists of the following sequential stages:

1. **Dataset Loading:** Load the TruthfulQA dataset (817 questions) with ground truth and wrong answers from `data/raw/truthfulqa_dataset.json`.
2. **Response Collection:** Use the OpenAI Batch API to collect:
 - 817 original responses (one per question).
 - $\leq 4,085$ variant responses (up to 5 variants $\times$ 817 questions).
 - $\leq 4,085$ variant text generations (Morpher output).

Batch processing provides 50% cost savings compared to synchronous API calls.

3. **Label Computation:** Compute binary labels (0 = truthful, 1 = hallucination) for all 817 questions by comparing the LLM’s extracted answer to the ground truth using the `microsoft/deberta-large-mnli` labeling model (Section 4.1.4).
4. **Feature Pre-Computation on GPU:** Run all NLI inferences in batch mode on the RTX 5070 with FP16 precision to compute:
 - All pairwise NLI scores between variant responses.
 - Self-verification contradiction scores.
 - Minority penalties via answer extraction and majority voting.
 - Pairwise NCD and average token entropy.

All features are cached in memory, eliminating repeated GPU computation during optimization.

5. **Bayesian Optimization:** Execute 500 Optuna trials with the TPE sampler to identify optimal fusion weights, decision threshold, and NLI parameters. Each trial evaluates a candidate configuration across all 817 questions using the cached features.
6. **Evaluation:** Apply the best-found configuration to compute precision, recall, F1-score, and accuracy. Generate confusion matrix for detailed error analysis.
7. **Ablation Study:** Repeat the optimization for each of the 8 signal combinations (Table 4.16) using 500 trials per combination. Compare F1, precision, and recall across configurations to quantify the marginal contribution of each new signal.

4.10. Reproducibility

To facilitate reproducibility, the following measures are in place:

- All configuration parameters are stored in version-controlled YAML files (`config/config.yaml` and `config/prompts.yaml`).
- Optimization results, including best parameters and full trial histories, are persisted in `config/optimization_results/` as JSON files.
- The Optuna TPE sampler is seeded (`seed = 42`) for the ablation study to ensure deterministic trial ordering.
- Batch API results are stored as organized JSON files in `data/batch_results/`, enabling re-execution of the optimization without additional API calls.
- The complete source code is available in the project repository.

4.11. Summary

This chapter has presented the complete experimental setup for evaluating the LOGIC-HALT system. The implementation employs a modular Python architecture built on PyTorch, Hugging Face Transformers, and Optuna, running on an NVIDIA RTX 5070 GPU with FP16 mixed precision. The primary evaluation dataset is TruthfulQA (817 questions), with labels computed via a separate NLI model to avoid data leakage. Response collection leverages the OpenAI Batch API for 50% cost reduction. A pre-computation strategy decouples GPU inference from the Bayesian optimization

loop, enabling 500 trials to complete in under one minute. The ablation study systematically evaluates all $2^3 = 8$ combinations of three newly proposed signals. A Flask-based web interface provides real-time demonstration with two-phase detection. The next chapter presents the experimental results and their analysis.

5. RESULTS AND DISCUSSION

This chapter presents the experimental results of the LOGIC-HALT system on the TruthfulQA benchmark (817 questions), analyzes the impact of the ground truth correction, reports the ablation study findings, and discusses limitations.

5.1. Ground Truth Contradiction: Discovery and Correction

During the development of LOGIC-HALT, we discovered a critical methodological issue in the original evaluation pipeline. The system’s primary signal—termed “GT Contradiction”—was computed by comparing the LLM’s response against the known ground-truth answer using NLI:

$$\text{GT_Risk} = 0.7 \cdot P_{con}(R, GT) + 0.3 \cdot (1 - P_{ent}(R, GT)) \quad (5.1)$$

where P_{con} and P_{ent} denote the contradiction and entailment probabilities from the NLI model, R is the LLM response, and GT is the ground-truth answer.

This signal accounted for 63.8% of the fusion weight in the original 4-signal system and was the primary driver of the reported metrics (F1 = 0.773, Precision = 0.977). However, the GT Contradiction signal requires knowledge of the correct answer at inference time—which is precisely what the system is supposed to determine. In any real-world deployment, the correct answer is not available; if it were, a hallucination detector would be unnecessary.

5.1.1. Impact of the Correction

Table 5.1 compares the original GT-based system with the corrected 5-signal system.

Table 5.1: Impact of removing ground truth dependency.

System	Signals	Precision	Recall	F1	GT Required
Original (GT-based)	4	0.977	0.640	0.773	Yes
Corrected (5-signal, precision-balanced)	5	0.602	0.917	0.727	No

The corrected system achieves an F1 score of 0.727 (precision-balanced optimization, 1000 trials)—a reduction of 0.046 from the GT-based system—while operating entirely without ground-truth information at inference time. Notably, recall increased from 0.640 to 0.917, meaning the corrected system detects substantially more actual hallucinations, though at the cost of reduced precision ($0.977 \rightarrow 0.602$). When optimized purely for F1 without precision constraints, the system achieves $F1 = 0.735$ but with $\text{recall} = 1.000$ and $\text{precision} = 0.581$, essentially flagging all responses as hallucinations.

5.2. Five-Signal System Results

5.2.1. Standard F1 Optimization

Using Bayesian optimization with 500 trials, the system achieved the following results on TruthfulQA (817 questions):

Table 5.2: Five-signal system performance (F1-optimized, 500 trials).

Metric	Value
Total Questions	817
Hallucinations in Dataset	472 (57.8%)
F1 Score	0.7346
Precision	0.5806
Recall	1.0000
Accuracy	0.5826
True Positives	472
False Positives	341
False Negatives	0
True Negatives	4

The system achieves perfect recall (1.000) but low precision (0.581), indicating that the optimizer found the “flag everything” strategy to maximize F1 given the dataset’s class distribution (57.8% hallucinations).

5.2.2. Precision-Balanced Optimization

To address the precision deficit, we re-optimized with a precision-penalized objective function that discounts F1 when precision falls below 0.60:

$$\text{Objective} = \begin{cases} F1 \cdot \frac{P}{0.60} & \text{if } P < 0.60 \\ F1 \cdot (1 + 0.15 \cdot \min(P - 0.60, 0.35)) & \text{otherwise} \end{cases} \quad (5.2)$$

Table 5.3: Precision-balanced optimization results (1000 trials).

Metric	F1-Optimized	Precision-Balanced
Precision	0.581	0.602
Recall	1.000	0.917
F1	0.735	0.727

Precision-balanced optimization yields a modest improvement in precision (0.581 $\rightarrow$ 0.602) with a controlled decrease in recall (1.000 $\rightarrow$ 0.917).

5.3. Bayesian Optimization Results

5.3.1. Optimized Signal Weights

Table 5.4 presents the Bayesian-optimized weights for the 5-signal fusion system.

Table 5.4: Optimized 5-signal fusion weights (500 trials, TruthfulQA).

Signal	Description	Weight
Minority Penalty (w_5)	Answer-level majority voting	0.3919
Inconsistency (w_2)	NLI-based self-consistency	0.2120
NCD (w_4)	Pairwise compression distance	0.1616
Entropy (w_3)	Token-level uncertainty	0.1493
Self-Verification (w_1)	Original vs. variant NLI	0.0852
Decision Threshold (τ)		0.2848

A surprising finding is that **Minority Penalty** emerges as the dominant signal with 39.19% of the total weight. This indicates that answer-level majority voting—a relatively simple signal based on regex-based answer extraction—is more discriminative than NLI-based consistency analysis (21.20%) or information-theoretic metrics (entropy 14.93%, NCD 16.16%). Self-Verification receives the lowest weight (8.52%), suggesting that comparing original and variant responses via NLI provides limited

additional information beyond what the consistency analysis already captures.

5.3.2. Optimization Convergence

The Bayesian optimizer converged rapidly, achieving near-optimal F1 within the first 100 trials:

Table 5.5: Optimization convergence.

Trial	Best F1
Trial 3	0.7324
Trial 50	0.7340
Trial 114	0.7346
Trial 500	0.7346 (converged)

The rapid convergence demonstrates the efficiency of the TPE sampler for this optimization problem.

5.4. Ablation Study

To evaluate the contribution of each signal, we conducted an ablation study testing three additional signals across all 8 combinations (500 trials per configuration):

1. **Negation Contradiction (Neg):** NLI entailment between original and negation-variant responses. If the model agrees with both a claim and its negation, this indicates logical inconsistency.
2. **Factual Token Variance (FTV):** Jaccard distance between factual tokens (numbers, proper nouns) across response variants.
3. **Semantic Answer Matching (SAM):** Sentence-BERT based semantic clustering of extracted answers for improved majority voting.

5.4.1. Ablation Analysis

The ablation study reveals several important insights:

1. **Negation Contradiction is the only signal that improves precision.** Adding the Neg signal raises precision from 0.578 to 0.601–0.606 across all configurations. This is because negation probing tests logical coherence rather than mere

Table 5.6: Ablation study results (sorted by F1, 500 trials each).

Configuration	Precision	Recall	F1
SAM	0.5791	1.000	0.7335
Baseline	0.5777	1.000	0.7324
FTV	0.5777	1.000	0.7324
FTV+SAM	0.5777	1.000	0.7324
Neg	0.6008	0.915	0.7254
Neg+FTV	0.5994	0.907	0.7218
Neg+SAM	0.6058	0.892	0.7215
Neg+FTV+SAM	0.6042	0.854	0.7076

consistency—a model that agrees with both a claim and its negation is exhibiting a qualitatively different type of failure.

2. **Factual Token Variance (FTV) provides no measurable benefit.** This is likely because TruthfulQA questions are predominantly textual/conceptual rather than numerical, limiting the applicability of number and proper noun comparison.
3. **Semantic Answer Matching (SAM) provides marginal improvement.** The improvement from string-based to semantic answer matching is minimal (F1: $0.7324 \rightarrow 0.7335$), suggesting that regex-based answer extraction already captures the majority of answer-level variation.
4. **Combining all three signals degrades performance.** The Neg+FTV+SAM configuration yields the lowest F1 (0.708), indicating that adding non-discriminative signals introduces noise that hampers optimization.

5.5. Threshold Analysis

Table 5.7 presents the precision-recall trade-off at different decision thresholds using the optimized weights.

The threshold sweep reveals that precision does not monotonically increase with threshold. At $\tau = 0.575$, precision reaches 0.750 but recall drops to 1.3%, rendering the system impractical. The optimal operating point depends on the intended application:

- **High-recall screening** ($\tau \leq 0.30$): Catches $\geq 98.5\%$ of hallucinations with $\sim 58\%$ precision. Suitable for automated pre-filtering before human review.
- **Balanced operation** ($\tau \approx 0.40$): Precision $\sim 58\%$ with recall $\sim 55\%$. Moderate performance in both dimensions.

Table 5.7: Performance at different thresholds (optimized weights).

Threshold	Precision	Recall	F1	TP	FP	FN
0.250	0.579	1.000	0.734	472	343	0
0.300	0.578	0.985	0.728	465	340	7
0.325	0.566	0.888	0.691	419	321	53
0.350	0.563	0.756	0.646	357	277	115
0.400	0.583	0.545	0.563	257	184	215
0.450	0.626	0.362	0.459	171	102	301
0.500	0.559	0.150	0.237	71	56	401
0.575	0.750	0.013	0.025	6	2	466

- **High-precision flagging** ($\tau \geq 0.50$): Precision increases but recall drops sharply. Only suitable for very high-confidence detections.

5.6. Signal Importance Analysis

5.6.1. Why Minority Penalty Dominates

The dominance of the Minority Penalty signal (39.19% weight) can be understood through the nature of hallucination manifestation. When an LLM hallucinates on a question, the generated variants (paraphrases, negations, etc.) often elicit different specific answers because the model lacks a stable factual foundation. The majority voting mechanism directly captures this answer-level disagreement without requiring sophisticated NLI analysis.

In contrast, NLI-based consistency (21.20%) analyzes the semantic relationship between full response texts, which can be confounded by stylistic variation. The higher weight of a simpler signal suggests that answer extraction followed by majority voting provides a more reliable indicator than full-text NLI comparison.

5.6.2. The Consistency-Correctness Gap

A fundamental insight from our experiments is that **all five signals measure variations of consistency, but none measure correctness**. Specifically:

- Self-Verification: Does the model agree with its own rephrased answer?
- Inconsistency: Do variant responses contradict each other?
- Entropy: How uncertain is the model during generation?

- NCD: How structurally similar are the responses?
- Minority Penalty: Do the extracted answers agree?

Without access to ground truth or external knowledge sources, the system cannot detect “confident hallucinations”—cases where the model consistently and confidently produces the same incorrect answer across all variants. This explains the precision ceiling of approximately 60%: roughly 40% of non-hallucinated questions are falsely flagged because their risk scores overlap with those of genuinely hallucinated questions.

5.7. Discussion

5.7.1. LOGIC-HALT as a Screening Tool

Given the precision-recall characteristics, LOGIC-HALT is best suited as a **high-recall screening tool** rather than a standalone decision system. At the default operating point ($\tau = 0.285$):

- 91.5% of actual hallucinations are detected (high recall)
- 60.2% of flagged responses are genuinely hallucinated (moderate precision)
- Combined with human review of flagged responses, the system provides an effective pre-filtering mechanism

5.7.2. Comparison with Related Methods

While direct comparison is difficult due to different datasets and evaluation protocols, we note that:

- SelfCheckGPT [3] achieves $F1 \approx 0.70$ on its evaluation set using sampling-based consistency alone
- LOGIC-HALT achieves $F1 = 0.727$ (precision-balanced) by combining consistency with information-theoretic and answer-level signals
- Unlike SelfCheckGPT, LOGIC-HALT explicitly addresses the GT leakage problem and reports only inference-time metrics

5.8. Limitations

1. **Precision Ceiling:** Without external knowledge or ground truth, precision is bounded at approximately 60%. The system cannot determine factual correctness from consistency alone.
2. **Consistent Hallucinations:** When the model confidently produces the same incorrect answer across all variants, all consistency-based signals will indicate low risk, resulting in a false negative.
3. **Single Target LLM:** Results were obtained using GPT-4o-mini. Performance may differ with other LLMs due to varying response characteristics and hallucination patterns.
4. **Signal Overlap:** Self-Verification and Inconsistency signals are conceptually similar (both measure NLI-based agreement between responses), limiting the diversity of the signal space.
5. **Answer Extraction Brittleness:** The regex-based answer extraction in Module F may fail on complex or multi-part answers, degrading the Minority Penalty signal.
6. **Computational Overhead:** Generating multiple variant responses and performing pairwise NLI analysis introduces latency proportional to $O(n^2)$ in the number of responses, which may limit real-time applicability.
7. **Negation-Recall Trade-off:** While Negation Contradiction is the only signal that improves precision, it simultaneously reduces recall (from 1.000 to 0.915), indicating a fundamental tension between the two metrics in this signal space.

5.9. Summary

This chapter presented the experimental evaluation of LOGIC-HALT on the TruthfulQA benchmark. The discovery and correction of the GT Contradiction dependency led to a redesigned 5-signal system that reports honest, inference-time metrics. Under precision-balanced optimization, the system achieves $F1 = 0.727$ with $Precision = 0.602$ and $Recall = 0.917$, making it effective as a high-recall screening tool. The ablation study confirmed the dominance of answer-level majority voting and identified Negation Contradiction as the only additional signal that improves precision. The fundamental limitation—that consistency-based signals cannot measure

factual correctness—establishes a clear direction for future work incorporating external knowledge sources.

6. CONCLUSION

This chapter summarizes the contributions of this thesis, discusses the implications of the findings, and outlines directions for future research.

6.1. Summary of Contributions

This thesis presented LOGIC-HALT, a multi-signal hallucination detection framework for Large Language Model outputs. The system was designed, implemented, and evaluated with the explicit goal of operating without ground-truth answers at inference time. The main contributions are:

1. **Five-Signal Fusion Architecture:** We developed a modular pipeline combining five complementary signals—Self-Verification, Inconsistency (NLI-based), Token Entropy, Pairwise NCD, and Minority Penalty—through a Bayesian-optimized weighted fusion layer. Evaluation on the TruthfulQA benchmark (817 questions) yielded $F1 = 0.727$, $Precision = 0.602$, and $Recall = 0.917$ under precision-balanced optimization.
2. **Ground Truth Leakage Identification:** We identified and corrected a critical methodological flaw where the original system used ground-truth answers as a detection signal during evaluation. This signal (GT Contradiction) accounted for 63.8% of the fusion weight and produced artificially inflated metrics ($Precision = 0.977$). We replaced it with inference-time-available proxy signals and reported honest metrics.
3. **Bayesian Hyperparameter Optimization:** Using Optuna’s TPE sampler with 500 trials, we jointly optimized 13 hyperparameters across all system modules, achieving systematic and reproducible parameter selection on the 817-question TruthfulQA benchmark.
4. **Comprehensive Ablation Study:** We tested 8 signal combinations (3 additional signals $\times$ all combinations) to quantify the marginal contribution of each signal. The ablation revealed that Minority Penalty (answer-level majority voting) is the strongest single signal (39.19% weight), and that Negation Contradiction is the only additional signal that improves precision.
5. **Two-Phase Detection:** We implemented an efficient pipeline where a fast answer-level consistency check precedes full NLI and information-theoretic analysis, reducing both computational cost and style-induced false positives.

6. **GPU-Optimized Implementation:** The complete system runs on consumer hardware (NVIDIA RTX 5070, 12 GB VRAM) with CUDA optimizations including FP16 inference and batch NLI processing, and includes a Flask-based web interface for real-time analysis.

6.2. Key Findings

The experimental evaluation yielded several important insights:

1. **Answer-Level Voting Outperforms NLI Consistency:** The simplest signal—regex-based answer extraction with majority voting—proved more discriminative than the sophisticated NLI-based consistency analysis. This suggests that for hallucination detection, *what* the model says (the final answer) matters more than *how* it says it (the explanation text).
2. **Consistency $\neq$ Correctness:** All five signals measure variations of inter-response agreement. Without access to external knowledge or ground truth, the system cannot distinguish “wrong but consistent” responses from correct ones. This fundamental limitation caps precision at approximately 60%.
3. **Negation Probing Provides Unique Information:** Among the three additional signals tested, only Negation Contradiction improved precision (0.578 $\rightarrow$ 0.606). By testing whether the model agrees with both a claim and its logical negation, this signal captures a qualitatively different type of failure—logical incoherence rather than simple inconsistency.
4. **GT Leakage is a Real Risk:** The original 4-signal system’s reported Precision of 0.977 was unattainable in practice because it relied on ground truth at inference time. This serves as a cautionary finding for the broader hallucination detection community: evaluation metrics must reflect realistic deployment conditions.
5. **High Recall is Achievable:** Even without ground truth, the system detects 91.5% of hallucinations, making it highly effective as a screening tool when combined with human review.

6.3. Limitations

Several limitations should be acknowledged:

1. **Precision Ceiling:** The system cannot exceed $\sim 60\%$ precision without external knowledge sources or a stronger verification model. This limits standalone deployment in high-precision applications.

2. **Single Model Evaluation:** All experiments used GPT-4o-mini as the target LLM. Hallucination patterns may differ across models (GPT-4, Claude, Llama, Mistral), and cross-model generalization was not evaluated.
3. **Consistent Hallucinations:** When the model confidently and repeatedly produces the same wrong answer across all variants, all consistency-based signals fail to detect the hallucination.
4. **Computational Cost:** Generating 5 variants + 1 verification query requires 7 LLM API calls per question, plus pairwise NLI analysis ($O(n^2)$). This may be prohibitive for high-throughput applications.
5. **English-Only Evaluation:** The system was evaluated only on English-language questions (TruthfulQA). Performance on other languages is unknown.

6.4. Future Work

Based on the findings and limitations of this thesis, we identify several promising directions for future research:

6.4.1. Short-Term Improvements

1. **External Knowledge Integration:** Adding a retrieval-augmented signal that fact-checks key claims against Wikipedia or Wikidata could provide the “correctness” signal that is currently missing, potentially breaking the precision ceiling.
2. **Stronger Verification Model:** Using GPT-4o (instead of GPT-4o-mini) for self-verification could improve the quality of the verification signal, as larger models have better factual knowledge.
3. **Cross-Model Consistency:** Querying multiple LLMs (GPT-4o, Claude, Gemini) with the same question and comparing their answers would provide a stronger consistency signal than single-model variant analysis.

6.4.2. Medium-Term Extensions

1. **Multi-Benchmark Evaluation:** Extend evaluation to additional benchmarks including HaluEval, FELM, and domain-specific datasets (medical, legal) to assess generalization.

2. **Learned Fusion:** Replace the linear weighted fusion with a trained classifier (e.g., gradient-boosted trees or a small neural network) that can capture non-linear interactions between signals.
3. **Token-Level Confidence Analysis:** Instead of average entropy, analyze entropy patterns at factual claim positions (names, numbers, dates) to detect localized uncertainty spikes.

6.4.3. Long-Term Research Directions

1. **Self-Correction Integration:** Combine detection with automatic correction—when a hallucination is flagged, use the verification signal and variant responses to generate a corrected answer.
2. **Formal Verification:** For logical reasoning tasks specifically, integrate symbolic logic solvers to formally verify the validity of reasoning chains.
3. **Theoretical Foundations:** Develop formal bounds on the precision achievable by consistency-based methods as a function of signal diversity and model behavior, providing theoretical guidance for system design.

6.5. Concluding Remarks

Large Language Models represent a transformative technology, but their tendency to hallucinate remains a critical barrier to trustworthy deployment. This thesis contributes to addressing this challenge through LOGIC-HALT, a practical framework that detects hallucinations without requiring ground-truth answers at inference time.

Our work demonstrates both the potential and the limitations of consistency-based detection. On one hand, by combining five complementary signals through Bayesian-optimized fusion, we achieve high recall (91.5%) suitable for screening applications. On the other hand, the fundamental insight that consistency does not imply correctness establishes a clear boundary: without external knowledge sources, precision remains bounded.

Perhaps the most significant contribution of this thesis is methodological: the identification and correction of the ground truth leakage problem. We believe that honest reporting of inference-time metrics—rather than evaluation-time metrics that rely on unavailable information—should be a standard practice in hallucination detection research. We hope this work encourages the community to critically examine evaluation protocols and report metrics that accurately reflect real-world deployment performance.

The source code, evaluation results, and web interface are available for further research and development.

BIBLIOGRAPHY

- [1] T. Brown et al., “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [2] Z. Ji et al., “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [3] P. Manakul, A. Liusie, and M. J. Gales, “Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 9004–9017.
- [4] L. Huang et al., “A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions,” *arXiv preprint arXiv:2311.05232*, 2023.
- [5] S. Min et al., “Factscore: Fine-grained atomic evaluation of factual precision in long form text generation,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023, pp. 12 076–12 100.
- [6] T. Y. Chen, S. C. Cheung, and S. M. Yiu, “Metamorphic testing: A new approach for generating next test cases,” Technical Report HKUST-CS98-01, Hong Kong University of Science and Technology, Tech. Rep., 1998.
- [7] X. Xie, J. W. Ho, C. Murphy, G. Kaiser, B. Xu, and T. Y. Chen, “Testing and validating machine learning classifiers by metamorphic testing,” *Journal of Systems and Software*, vol. 84, no. 4, pp. 544–558, 2011.
- [8] M. Li and P. Vitányi, *An introduction to Kolmogorov complexity and its applications*. Springer, 2008, vol. 3.
- [9] R. Cilibrasi and P. M. Vitanyi, “Clustering by compression,” *IEEE Transactions on Information Theory*, vol. 51, no. 4, pp. 1523–1545, 2005.
- [10] S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning, “A large annotated corpus for learning natural language inference,” in *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, 2015, pp. 632–642.
- [11] P. He, X. Liu, J. Gao, and W. Chen, “Deberta: Decoding-enhanced bert with disentangled attention,” in *International Conference on Learning Representations*, 2021.

CURRICULUM VITAE

Personal Information

Name: Ömer Faruk ŞENER
Date of Birth: June 30, 1998
Place of Birth: İzmir, Turkey
Email: omer.sener@gtu.edu.tr

Education

2022 – 2026	B.Sc. in Computer Engineering Gebze Technical University, Kocaeli, Turkey Graduation Project: LOGIC-HALT Advisor: Dr. Öğr. Üyesi Tülay AYYILDIZ
2012 – 2016	High School Akhisar Anadolu Öğretmen Lisesi, Manisa, Turkey

Technical Skills

- **Programming Languages:** Python, JavaScript, Java, C/C++
- **Machine Learning:** PyTorch, TensorFlow, Hugging Face Transformers
- **Web Development:** Flask, React, HTML/CSS
- **Tools:** Git, Docker, Linux, LaTeX
- **Languages:** Turkish (Native), English (Advanced)

Projects

- **LOGIC-HALT (2024-2026):** Hallucination detection framework for Large Language Models using metamorphic testing and information theory.

Areas of Interest

- Natural Language Processing
- Large Language Models
- Machine Learning
- Software Testing
- Trustworthy AI

APPENDICES

Appendix A: Test Dataset

The complete test dataset used for evaluation consists of 20 questions across 10 categories. Below is a sample of questions with their expected classifications.

Table 6.1: Sample questions from the test dataset.

ID	Category	Question	Expected
hal_001	Factual	What is the capital of France?	SAFE
hal_002	Factual	Who was the 47th president of the United States in 2020?	HAL
hal_005	Mathematical	What is $15 + 27$?	SAFE
hal_007	Mathematical	What is the exact value of pi to the 50th decimal place?	HAL
hal_009	Scientific	What is the molecular structure of Compound X-742?	HAL
hal_016	Logical	If all A are B, and all B are C, then are all A also C?	SAFE
hal_017	Temporal	What major event happened on February 30, 2020?	HAL

Appendix B: System Configuration

6.5.0. B.1 Fusion Layer Parameters

Listing 6.1: Default configuration (config.yaml)

```
1 fusion:
2   alpha: 0.15    # Consistency weight
3   beta: 0.50     # Entropy weight
4   gamma: 0.35   # NCD weight
5   hallucination_threshold: 0.55
6
7 answer_validator:
8   minority_penalty: 0.30
9   min_responses: 3
```



```
10 high_confidence_threshold: 0.80
```

6.5.0. B.2 Response Generation Parameters

Listing 6.2: LLM query configuration

```
1 generation:
2   model: "gpt-4o-mini"
3   temperature: 0.3
4   max_tokens: 600
5   num_responses: 10
6   logprobs: true
7   top_logprobs: 5
```

Appendix C: API Endpoints

The web interface provides the following REST API endpoints:

Table 6.2: API endpoints.

Endpoint	Method	Description
/	GET	Main web interface
/analyze	POST	Analyze question for hallucinations
/health	GET	System health check
/statistics	GET	Get evaluation statistics

6.5.0. C.1 Example API Request

Listing 6.3: Example analysis request

```
1 import requests
2
3 response = requests.post(
4     "http://127.0.0.1:5001/analyze",
5     json={
6         "question": "What is the capital of France?",
7         "num_responses": 10
8     }
```

```

9 )
10
11 result = response.json()
12 print(f"Risk_Score:_{result['summary']['average_risk']}")
13 print(f"Hallucinations:_{result['summary']['hallucinations_detected']}")

```

Appendix D: Sample Output

6.5.0. D.1 Analysis Result Example

Listing 6.4: Sample analysis output

```

1 {
2   "success": true,
3   "question": "What_is_the_capital_of_France?",
4   "summary": {
5     "total_responses": 10,
6     "safe_count": 10,
7     "hallucinations_detected": 0,
8     "average_risk": 0.30
9   },
10  "responses": [
11    {
12      "id": 1,
13      "is_hallucination": false,
14      "risk_score": 0.30,
15      "category": "safe",
16      "extracted_answer": "paris",
17      "answer_is_majority": true,
18      "metrics": {
19        "consistency": 0.85,
20        "entropy": 0.25,
21        "ncd": 0.40
22      }
23    }
24  ]
25 }

```

Appendix E: Project Repository Structure

Listing 6.5: Project directory structure

```
1 logic-halt/
2 |-- src/
3 |   |-- morpher.py           # Variant generation
4 |   |-- interrogator.py      # LLM querying
5 |   |-- consistency.py       # NLI analysis
6 |   |-- complexity.py        # Entropy + NCD
7 |   |-- fusion.py            # Decision fusion
8 |   |-- answer_validator.py  # Answer extraction
9 |   |-- detector.py          # Main pipeline
10 |
11 |-- web/
12 |   |-- app.py               # Flask application
13 |   |-- templates/           # HTML templates
14 |
15 |-- config/
16 |   |-- config.yaml          # Parameters
17 |   |-- prompts.yaml         # LLM prompts
18 |
19 |-- data/
20 |   |-- raw/                  # Test datasets
21 |   |-- processed/            # Results
22 |
23 |-- thesis/                   # LaTeX thesis
24 |-- requirements.txt
25 |-- README.md
```